

Oracles, tokens and off-chain networks

Things build with smart contracts

Leander Jehl

Oracles

Oracle

Access data outside the blockchain

An **Oracle** is a smart contract that publishes information about real world data on the chain.

Oracle

Example: Rain insurance

- Insurance contract needs weather data to
 - Pay out policies
 - Determine prices



insurance contract

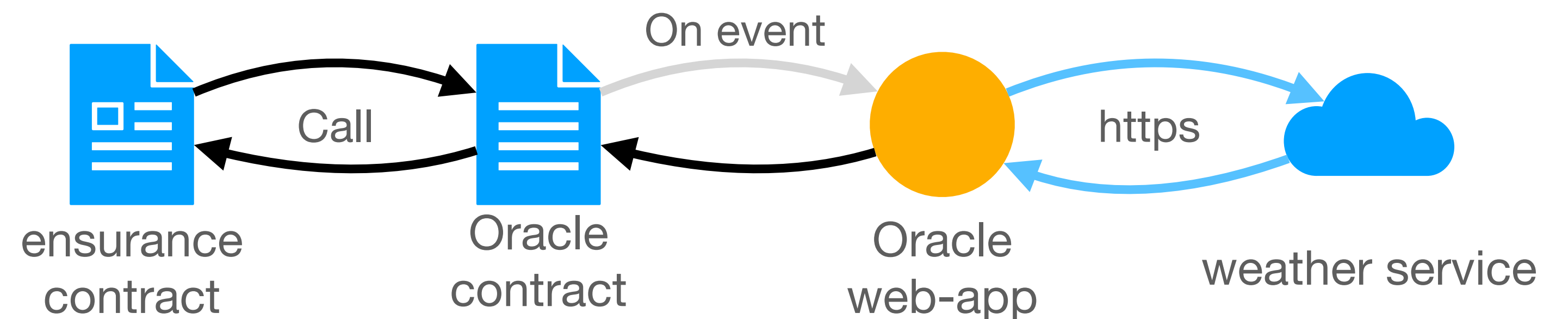


weather service (yr.no)

Oracle

Example: Rain insurance

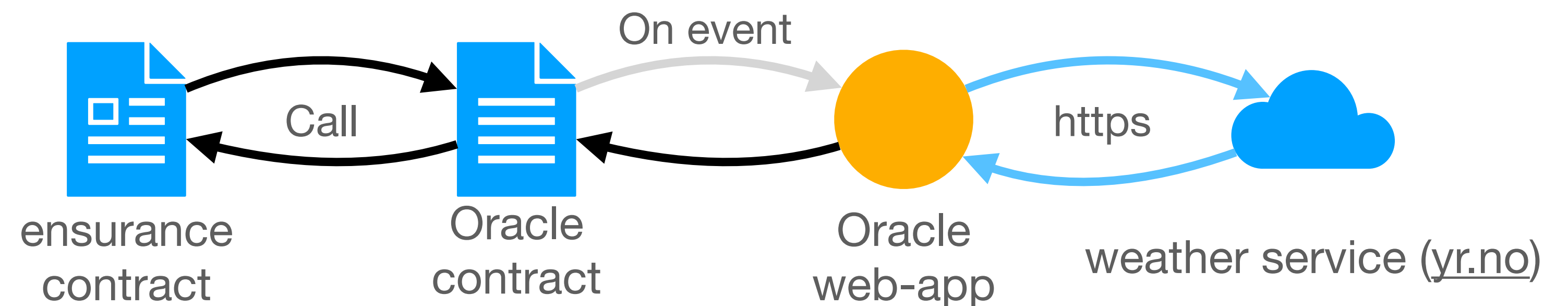
- Insurance contract needs weather data to
 - Pay out policies
 - Determine prices



Oracle

Example: Rain insurance

- Insurance contract calls oracle contract
- Oracle contract emits event
- Oracle web app listens to event
- Web app gets data from api
- Web app invokes contract

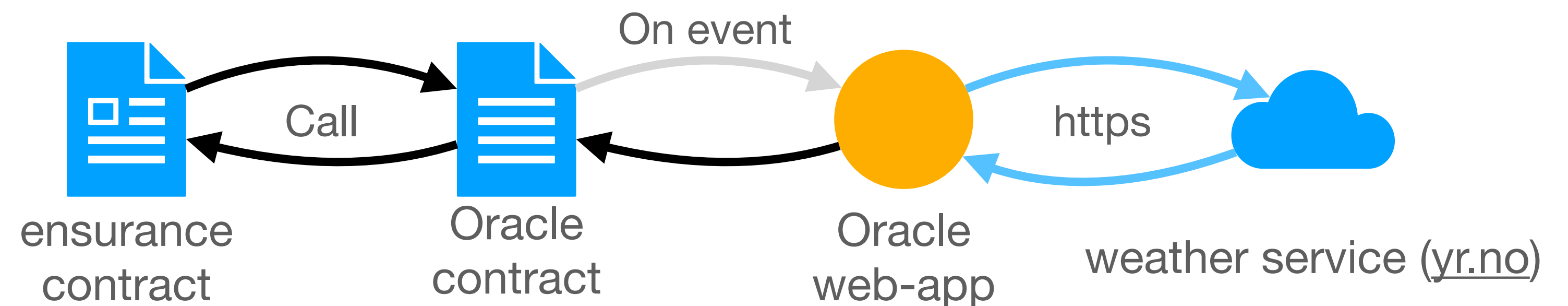


Oracle

Example: Rain insurance

- Insurance contract calls oracle contract
- Oracle contract emits event
- Oracle web app listens to event
- Web app gets data from api
- Web app invokes contract

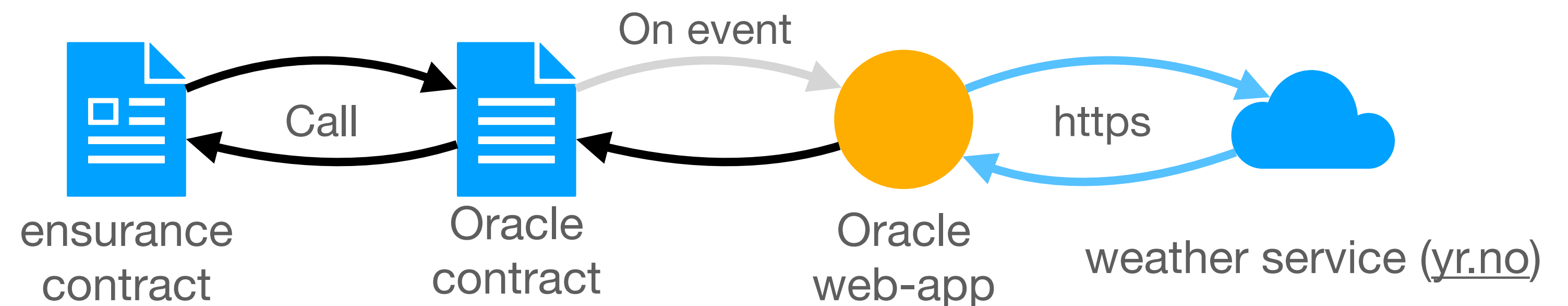
Check [cryptozombies tutorial](#)



Oracle

Example: Rain ensurance

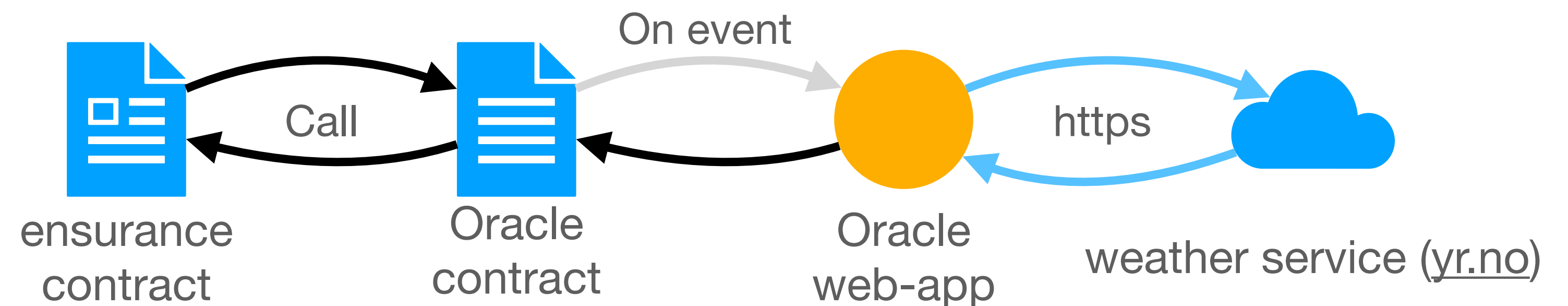
- Why should we use an extra oracle contract?
 - Can update if we need to update oracle
- Who do we need to trust?
 - Oracle provider, and API provider



Oracle

Example: Rain insurance

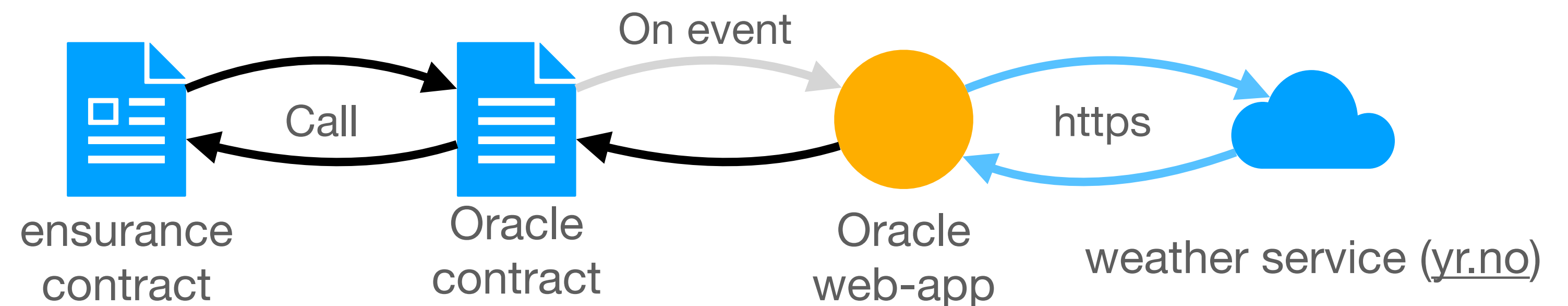
- Can we avoid trusting the oracle?
 - Yes, run oracle web-app in trusted execution (Intel SGX)



Oracle

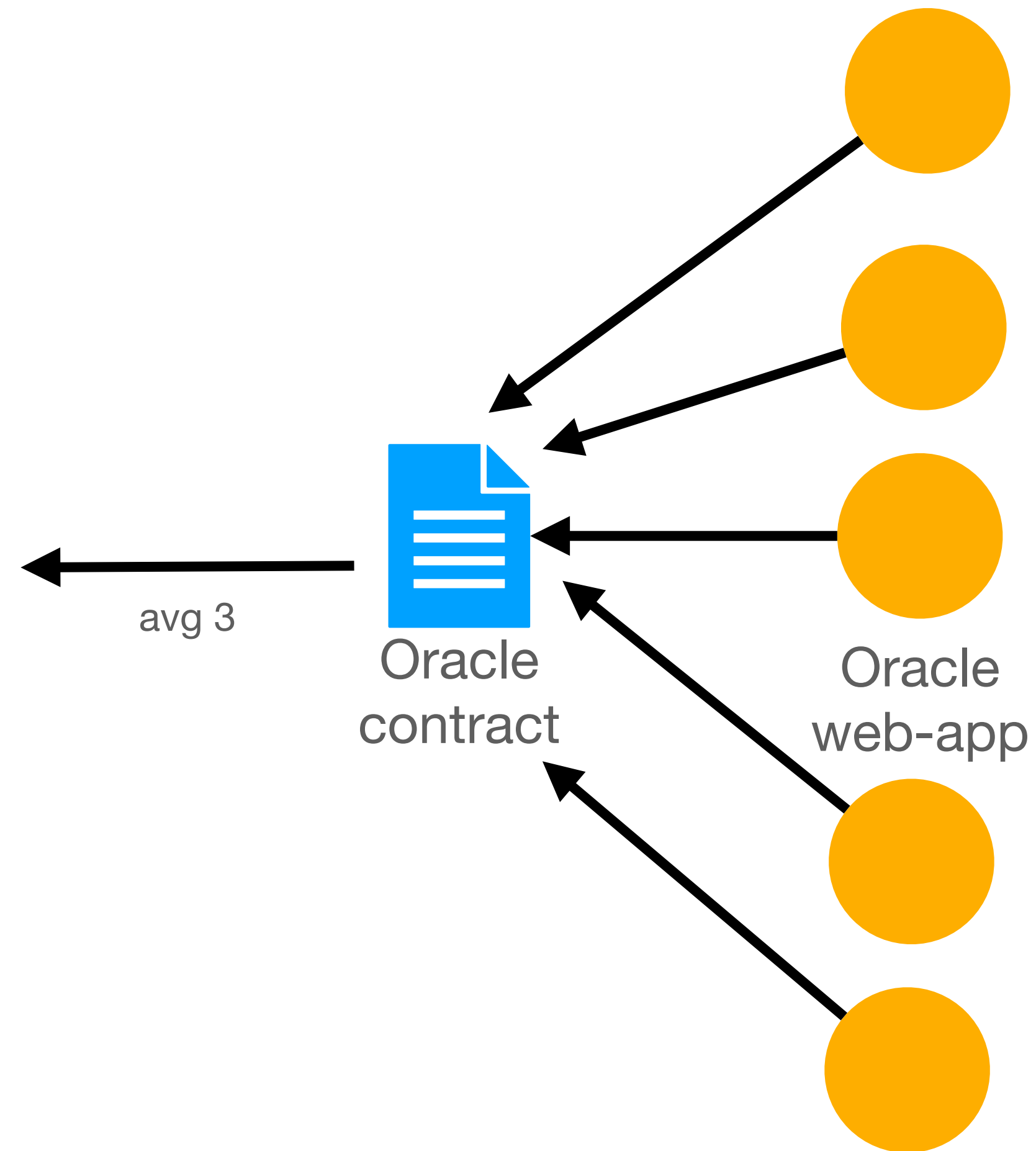
Variations

- Access private data, e.g. using login
 - Yes, run oracle web-app in trusted execution (Intel SGX)
- Provide oracle service, that anyone can use



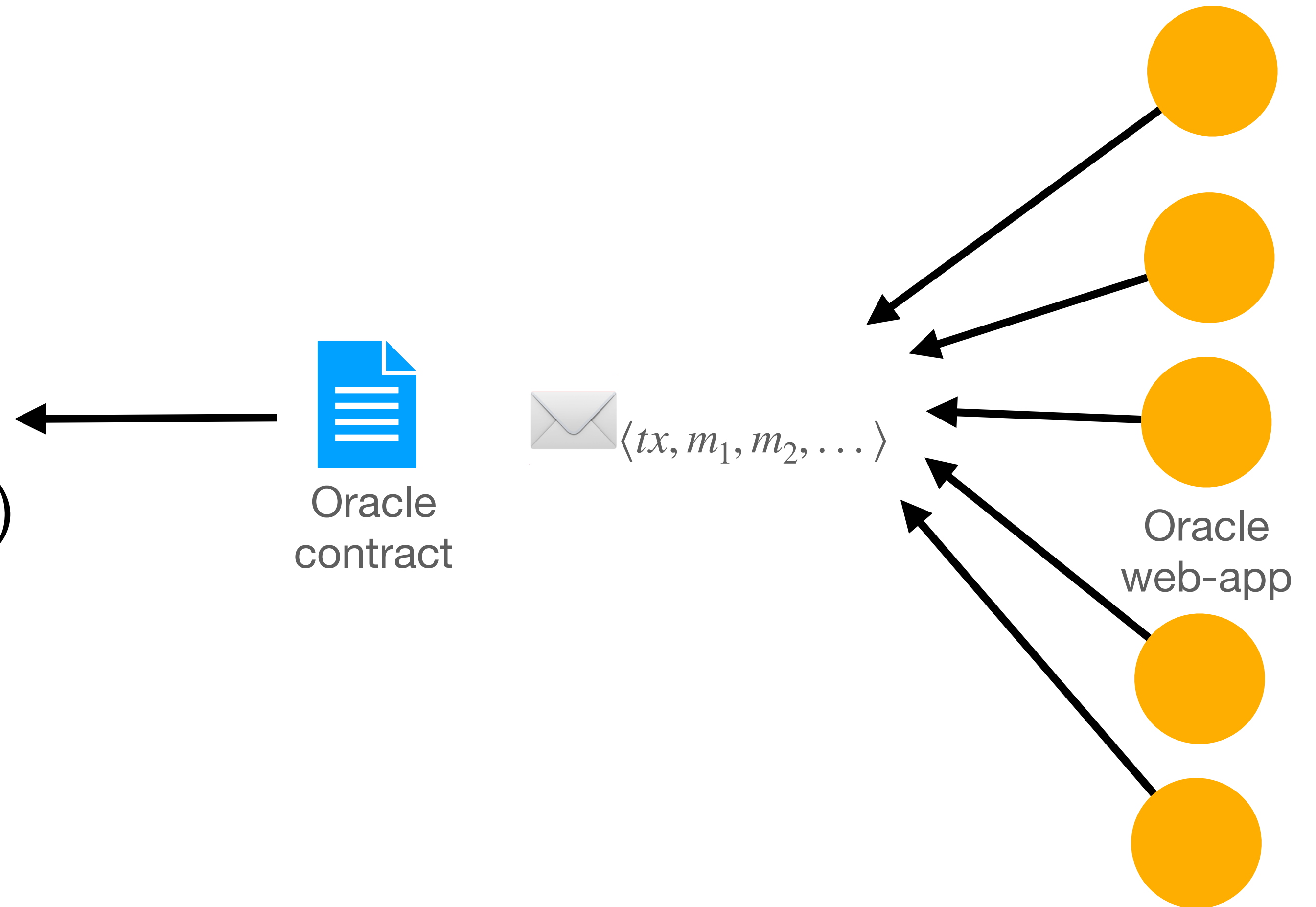
Oracle Variations

- Oracle aggregator
 - Multiple oracle providers
 - Aggregate input and remove outliers
 - Rewards for accurate input



Oracle Variations

- Oracle network
 - Multiple oracle participants
 - Coordinate in external structure (another blockchain)



Oracle

Another example

- Stable coin provider
 - Contract:
 - Send x tokens (ether) to get balance y on contract
 - Withdraw your balance by y , get x ether
- Price oracle
 - price of ETH in \$

Tokens

Tokens

Native and non-native tokens

- Native tokens is the base currency of a blockchain
 - Bitcoin, ether, ...
- Non-native tokens are similar but they are build using smart contracts

Tokens

Non-native tokens

- A smart contract keeps token balances
- Limited supply?
- Holders get benefits?
 - Voting rights
 - Discount
 - Etc

```
contract TokenBank {
    mapping(address => uint) private balances;
    address public owner;
    uint public price;

    // function SimpleBank() deprecated syntax for
    constructor(uint tprice) public {
        owner = msg.sender;
        price = tprice;
    }

    function buy() public payable returns(uint) {
        balances[msg.sender] += msg.value/price;
        return balances[msg.sender];
    }

    function transfer(uint amount, address receiver) public returns () {
        if (balances[msg.sender] >= amount){
            balances[msg.sender] -= amount;
            balances[receiver] += amount;
        }
    }

    function balance() view public returns (uint) {
        return balances[msg.sender];
    }
}
```

Tokens

Non-fungible tokens

- A smart contract keeps token balances
- Fungible tokens: All the same
- Non-fungible tokens: each token is different

```
contract NFToken {
    mapping(uint256 tokenId => address) private owners;
    address public owner;

    constructor() public {
        owner = msg.sender;
    }

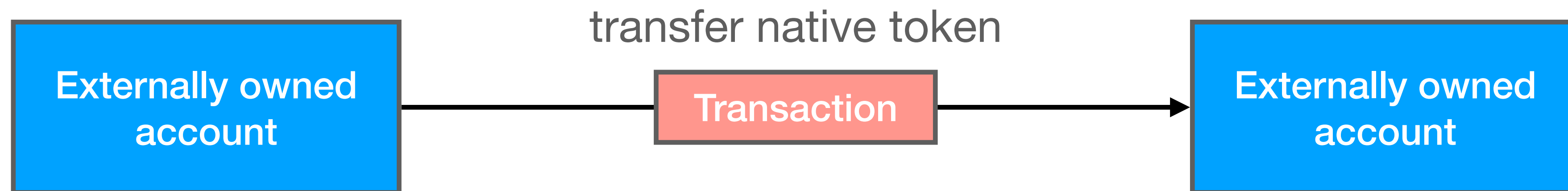
    function create(uint tid) public returns() {
        require(msg.sender == owner);
        require(owners[tid] == address(0));
        owners[tid] = owner;
    }

    function transfer(uint tid, address receiver) public returns () {
        if (owner[tid] == msg.sender) {
            owner[tid] = receiver;
        }
    }
}
```

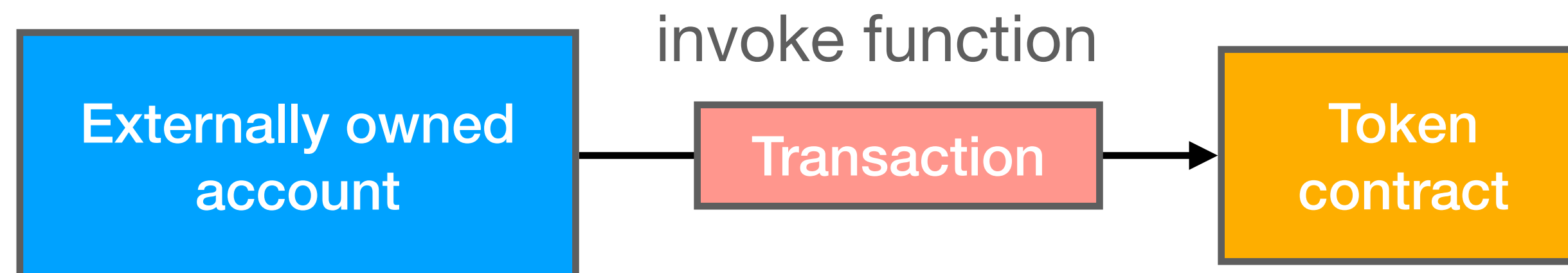
Tokens

Transfer tokens

- Transfer native token by transaction



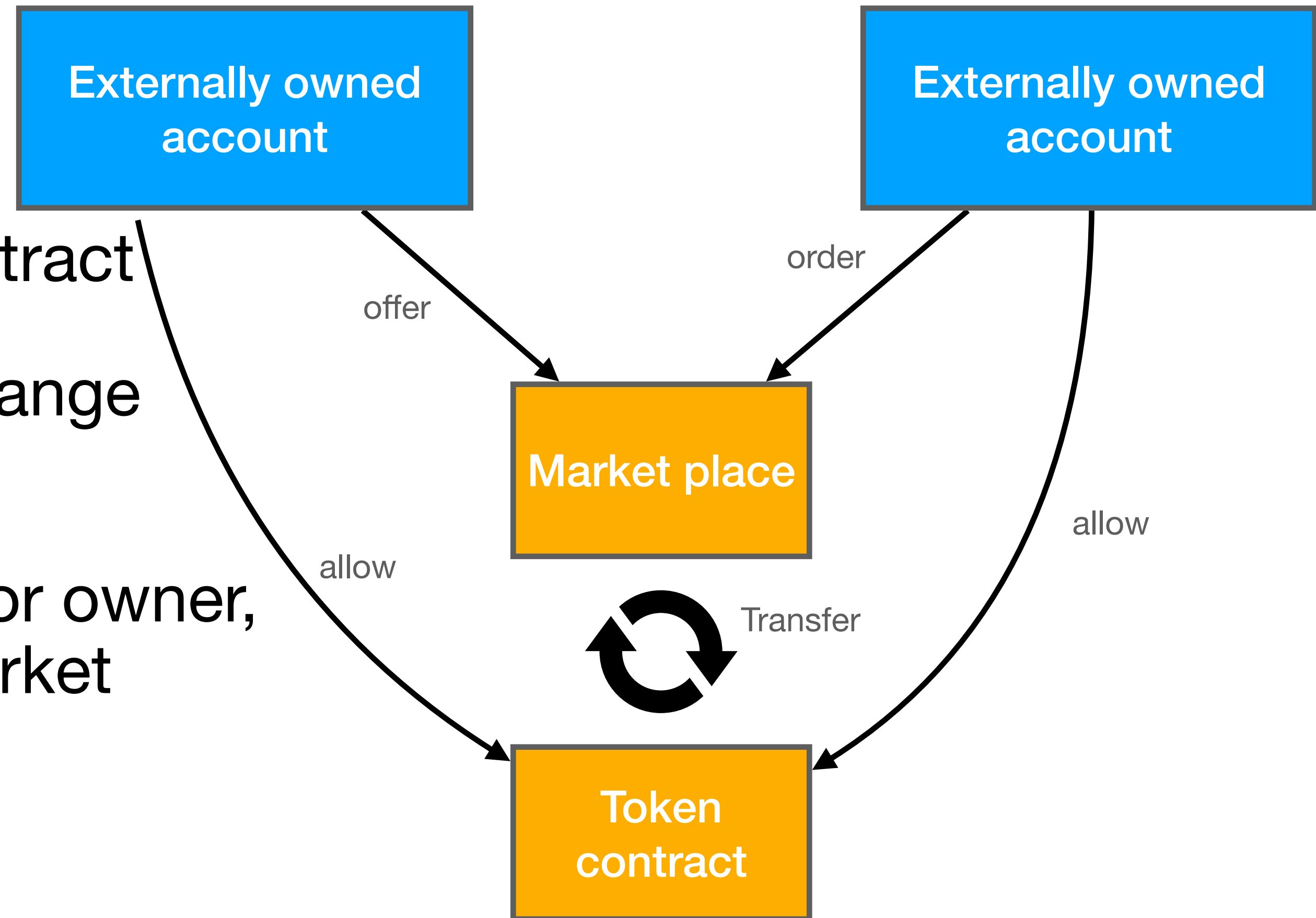
- Transfer non-native token by function call on token contract



Tokens

ERC20 Tokens

- Standard interface for token contract
- Allows markets to sell and exchange all compliant tokens
- Token contract needs support for owner, to allow/delegate transfer to market place



Token contract

<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v5.5.0-rc.1/contracts/token/ERC20/ERC20.sol>

Stable coins

Tokens with a stable value

- Designed for stable price 1token = 1\$
 - Keep reserve to back value of tokens
 - Fiat currency held in traditional banking (e.g. Tether, USDC, ...)
 - Different cryptocurrency locked in smart contracts (e.g DAI)
 - Custom investement token and mechanisms to affect the stable price, e.g. printing when too high, buy back when too low.

Stable coins

Tokens with a stable value

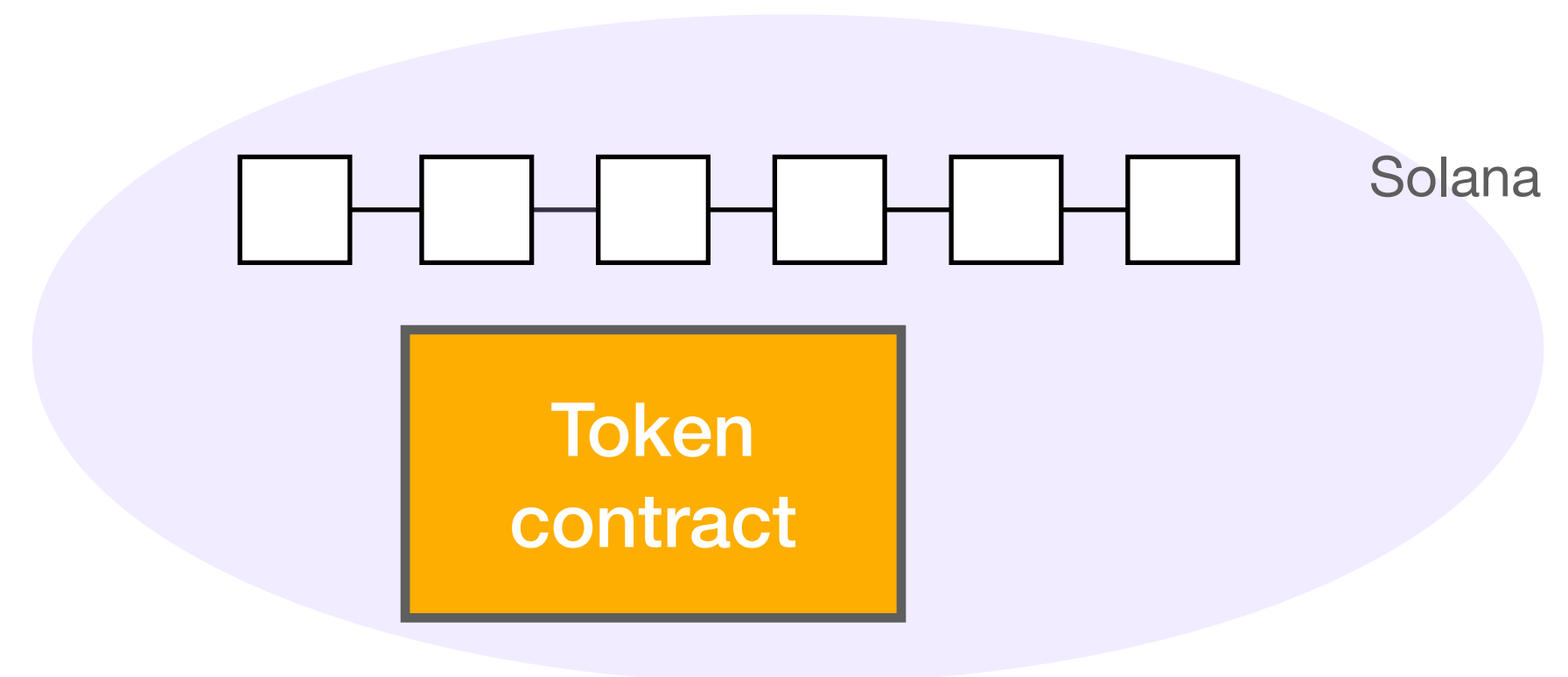
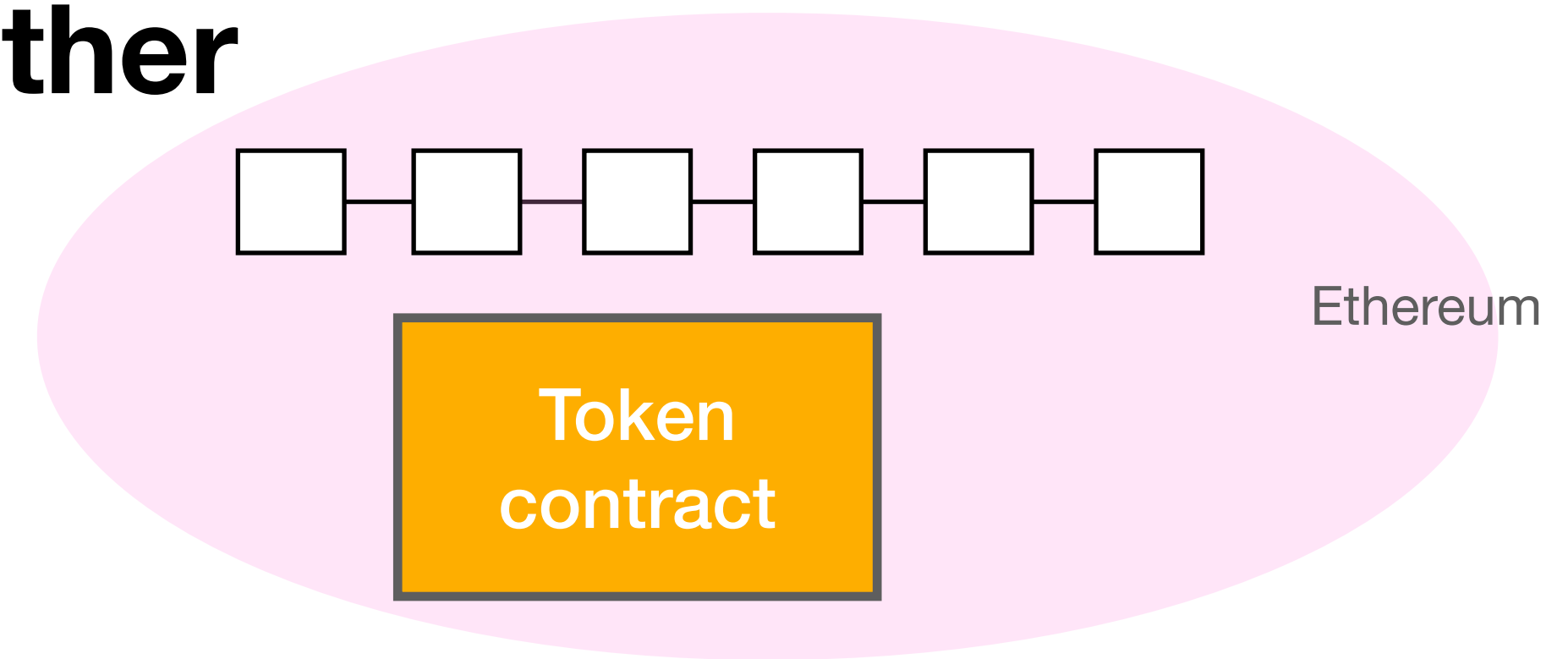
- Designed for stable price 1token = 1\$
 - Keep reserve to back value of tokens
 - Fiat currency held in traditional banking (e.g. Tether, USDC, ...)
Problem: Liquidity, and how the reserve is structured,
e.g. fiat, obligations, ...
 - Different cryptocurrency locked in smart contracts (e.g DAI)
Problem: strong price fluctuations in locked cryptocurrency
 - Custom investment token
Problem: what if investment token loses value?

Bridges

Bridges

Move tokens from one chain to another

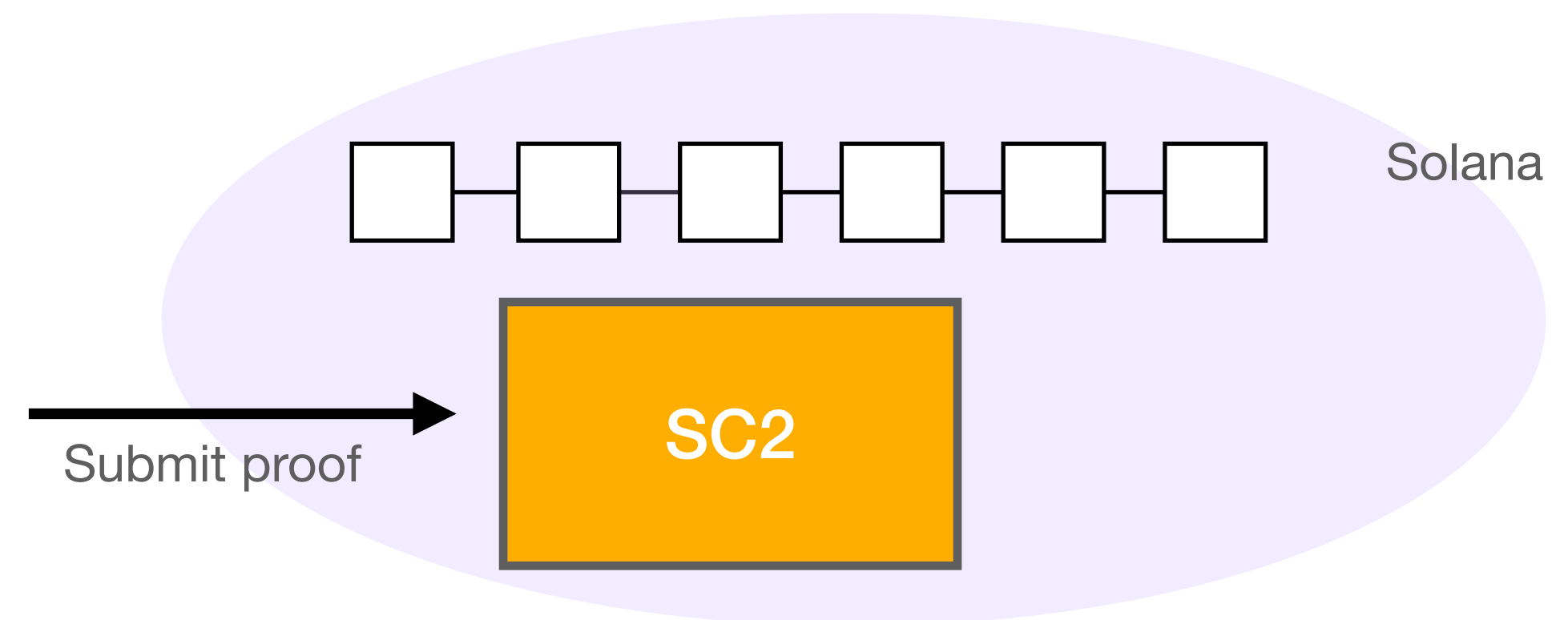
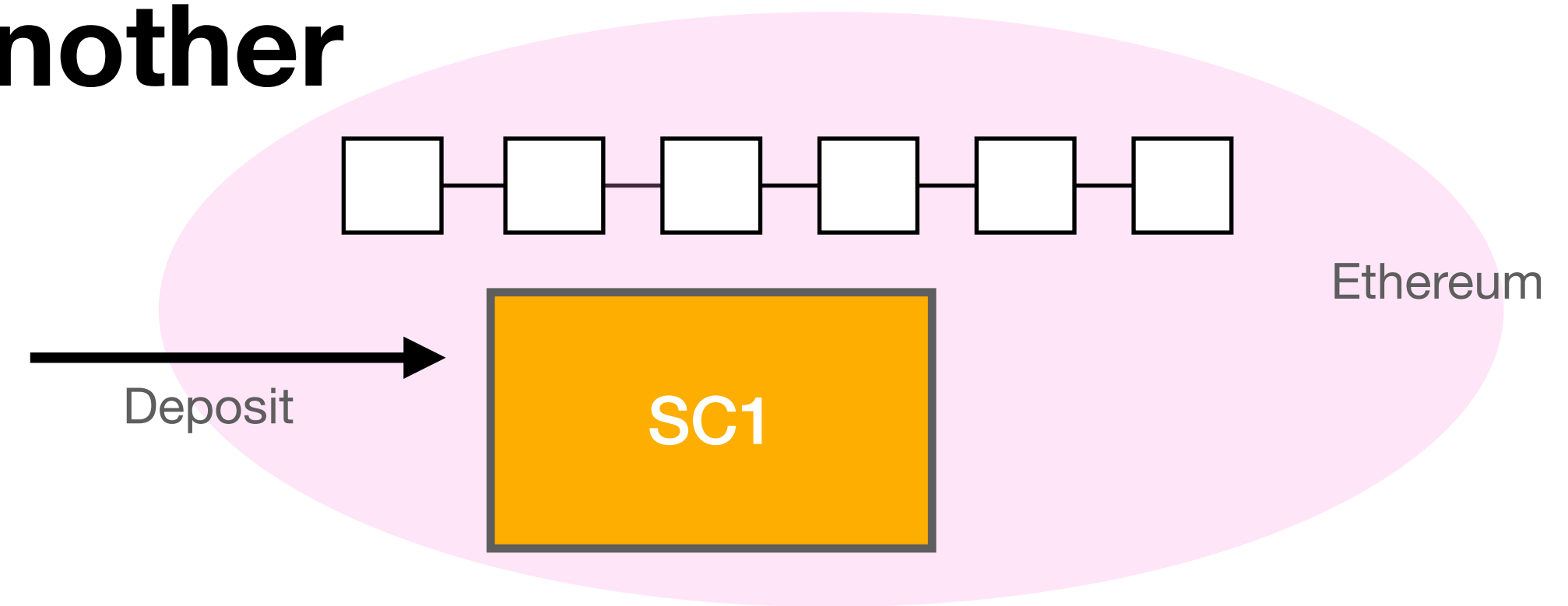
- Can you move an ERC20 token from one blockchain to the other?
- Can you move Ether from one blockchain to another?
- Can you do atomic swap accross different blockchains?
 - Swap Eth for SOL



Bridges

Move tokens from one chain to another

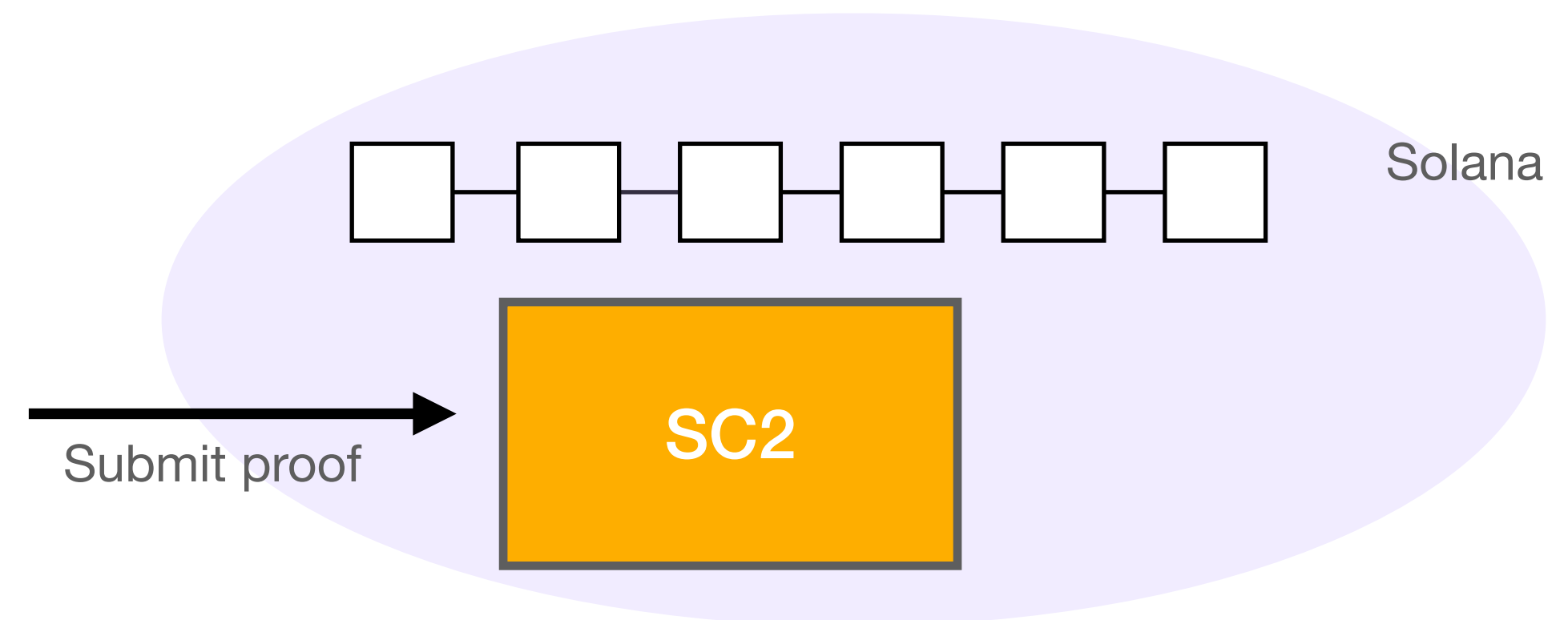
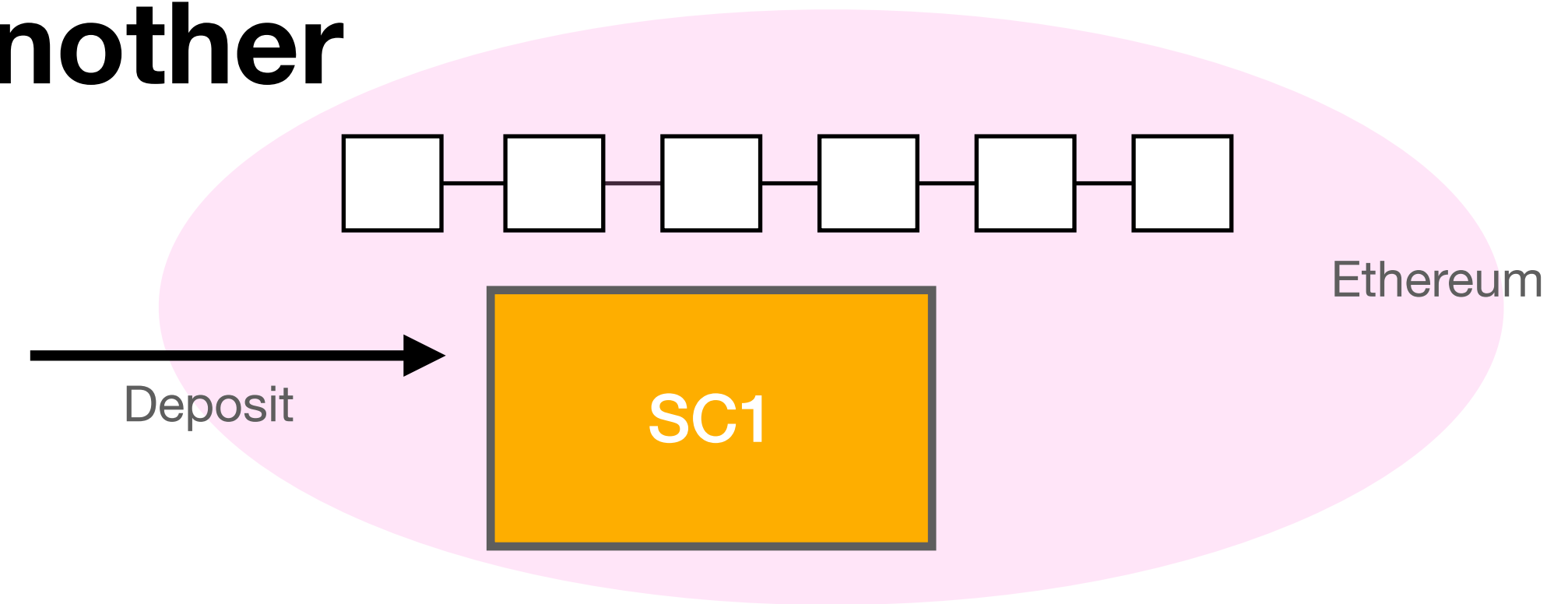
- Bridge: Deposit and mint:
 - Deposit Eth in SC1
 - Send proof of deposit to SC2
 - Create ERC20like - Eth token on Solana



Bridges

Move tokens from one chain to another

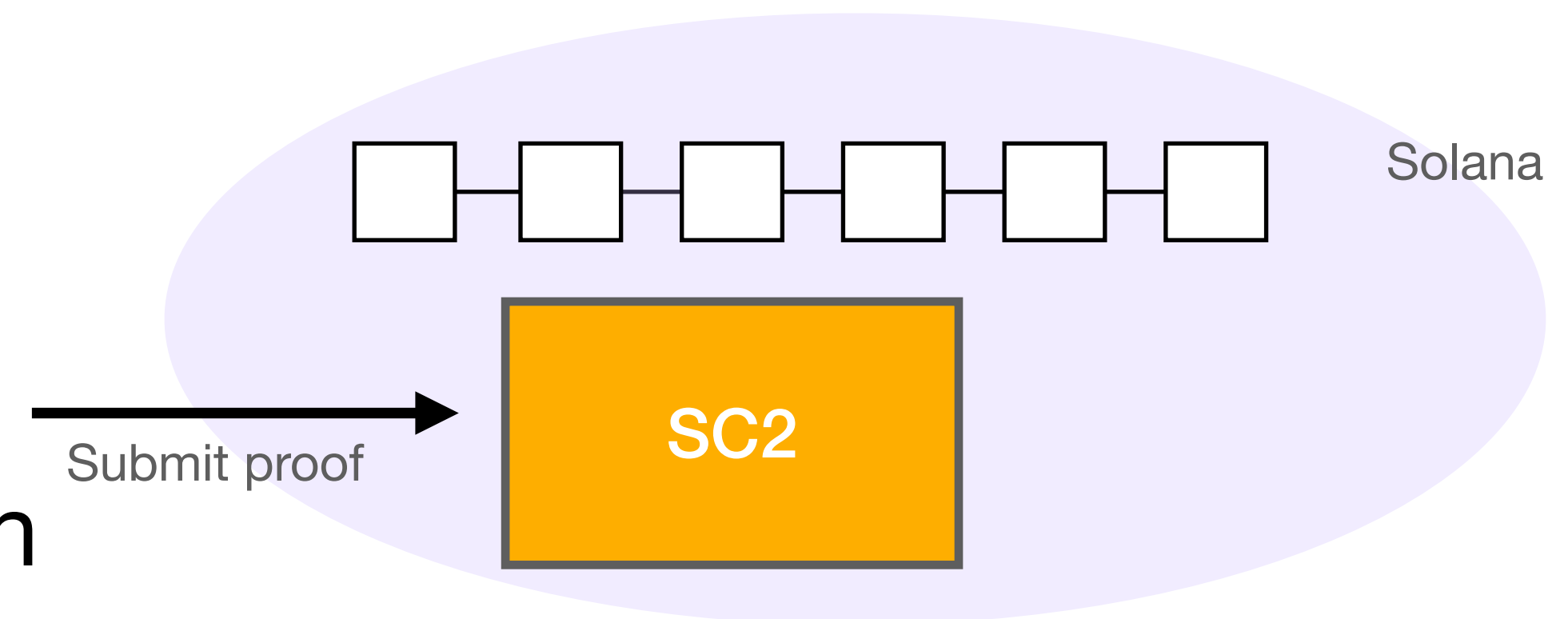
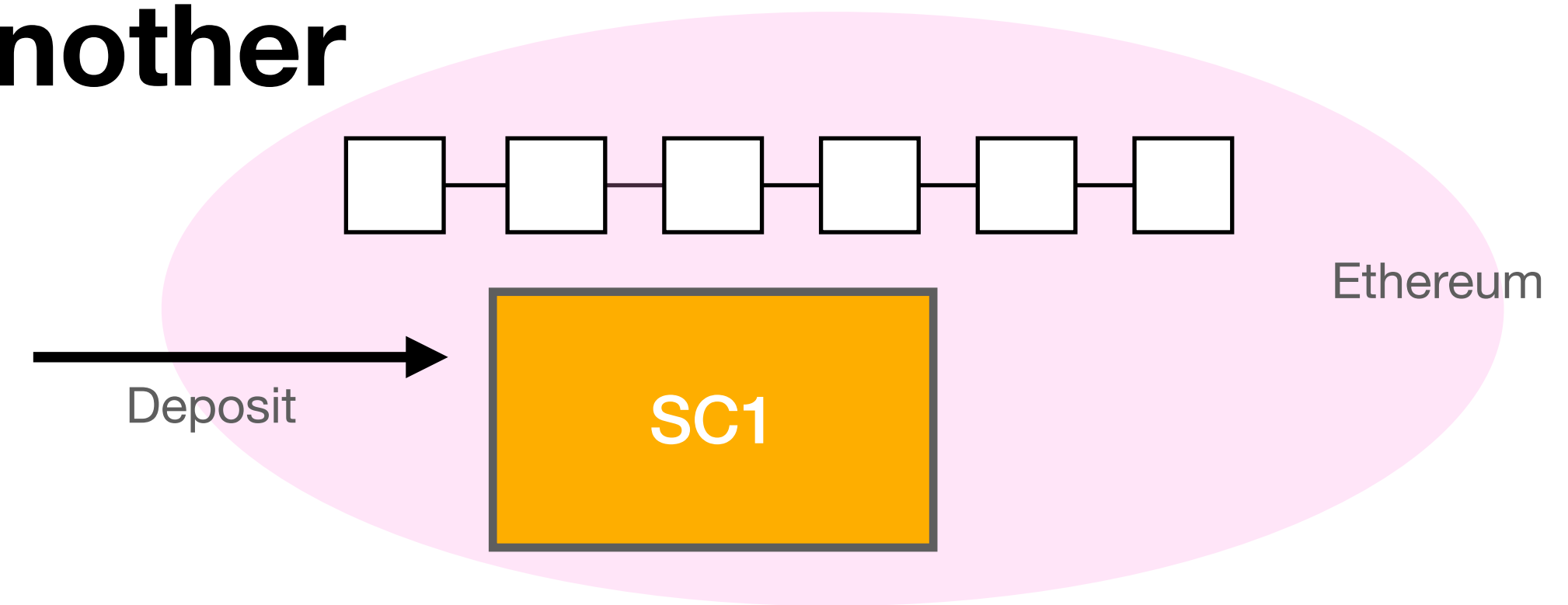
- Bridge: Deposit and mint:
 - What is a proof:



Bridges

Move tokens from one chain to another

- Bridge: Deposit and mint:
 - What is a proof:
 - Transaction submitted by owner (trust one guy)
 - Transaction signed by a trusted committee (trust multiple guys)
 - Merkle proof for deposit transaction or log statement



Off chain networks / Layer 2

Off-chain / Layer 2

General idea

- **Not all data needs to be on the chain.**
- **Save on transaction fees**
- **Get some guarantees from chain.**
 - **Payment channels**
 - **Rollups**

Payment channels

Payment channels

General idea

- **Idea:** If two parties agree, they can do a transaction outside of the chain without paying fees.
 - Once they disagree, they can use the chain to settle the dispute.
 - Can increase transaction throughput
 - Can give low fees

Payment Channels

Example: Uni-directional payment channel

- **Idea:** Allow any number of payments from A to B within given limit
- A creates contract with balance.
- A can send signed statements of B's balance to B
- B can cash in his balance with the contract
- If B does not cash in, A can terminate the contract and get back the balance, after expiration date.

Check example

<https://solidity-by-example.org/app/uni-directional-payment-channel/>

Payment Channels

Example: Bi-directional payment channel

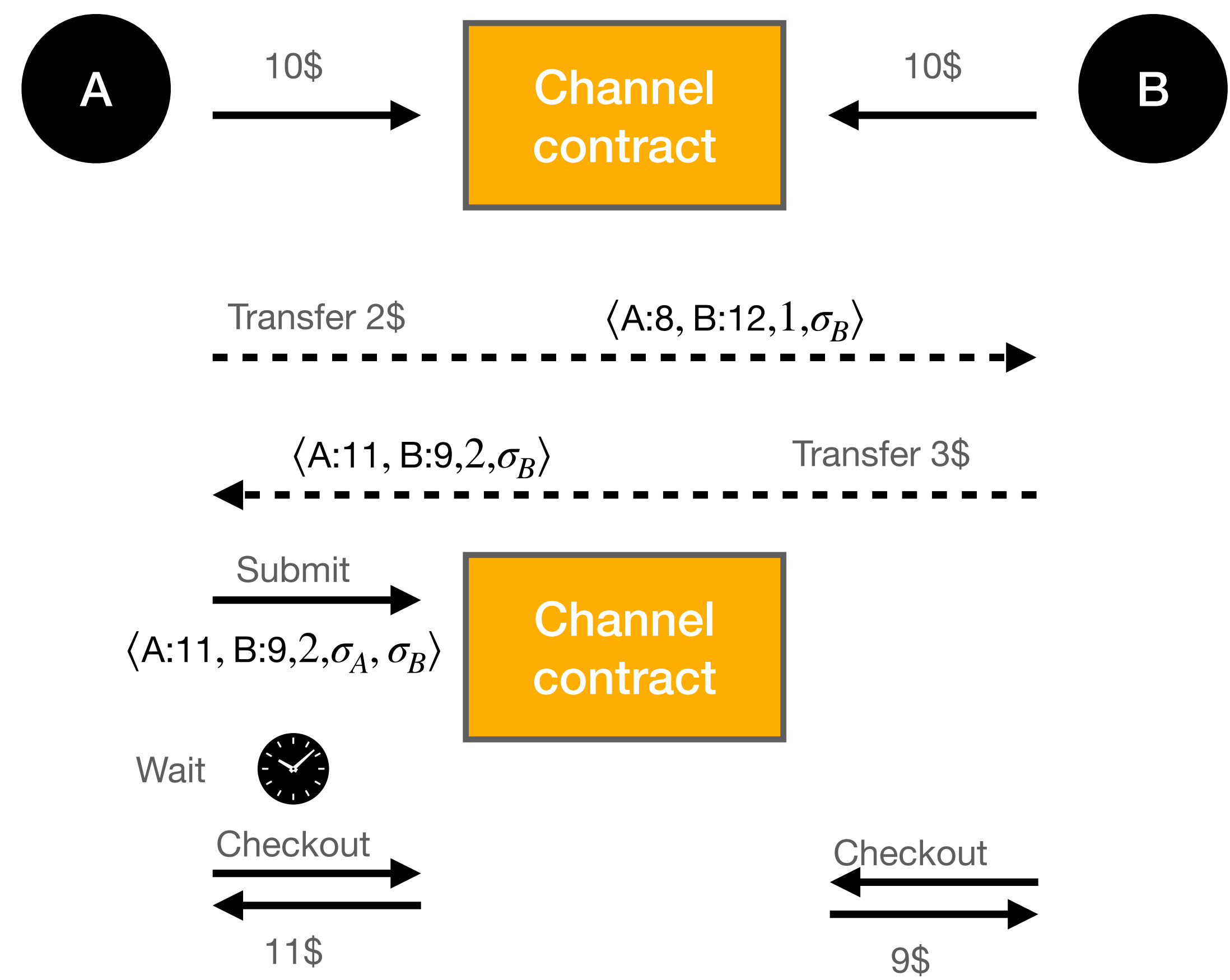
- **Idea:** Allow any number of payments between A and B within given limit,
- A and B both pay a balance to contract
- A and B can send signed statements of their balances to each other, with increasing nonces
- A or B can submit balance, signed by both to contract. This triggers countdown
- If other party does not submit a balance with larger nonce, balances are paid out.

Check example

<https://solidity-by-example.org/app/bi-directional-payment-channel/>

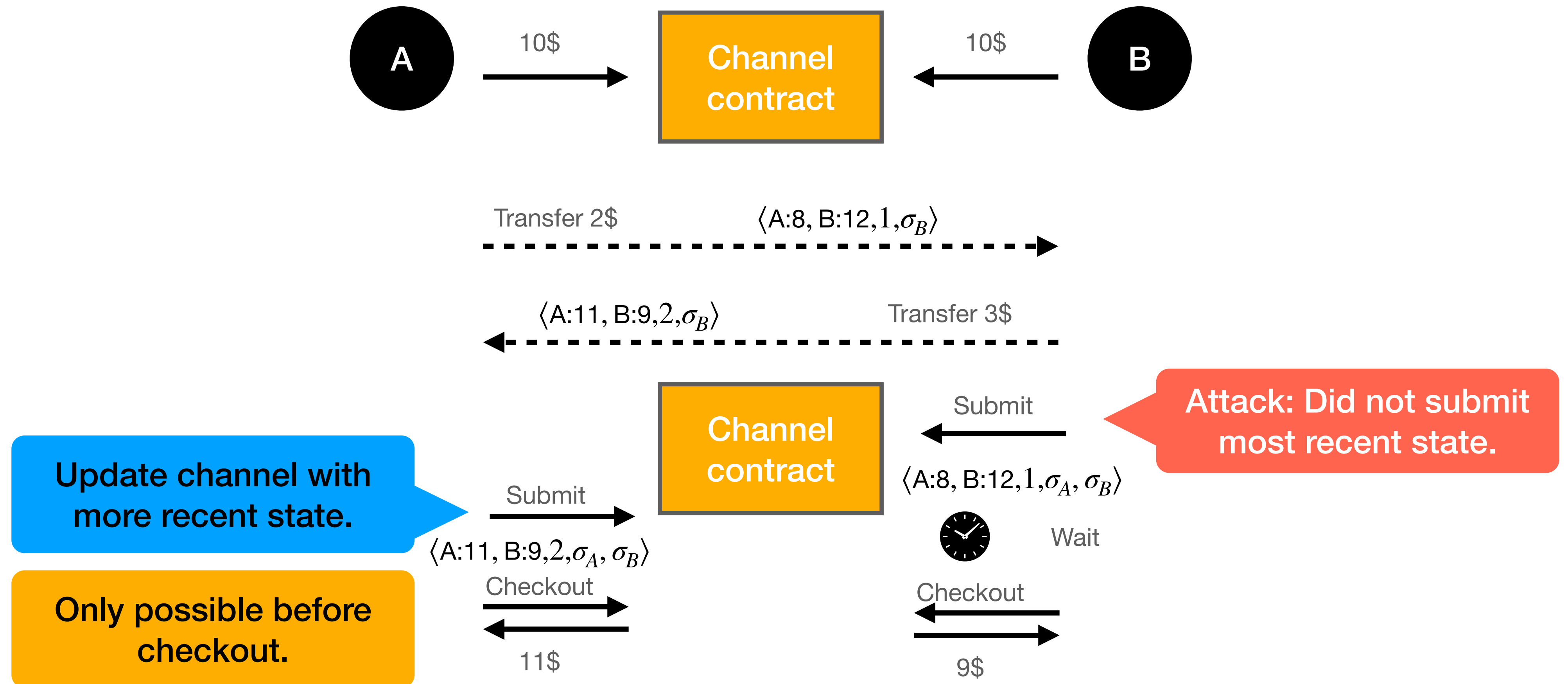
Payment Channels

Example: Bi-directional payment channel



Payment Channels

Example: Bi-directional payment channel



Payment Channels

Example: Bi-directional payment channel

- **Idea:** Allow any number of payments between A and B within given limit,

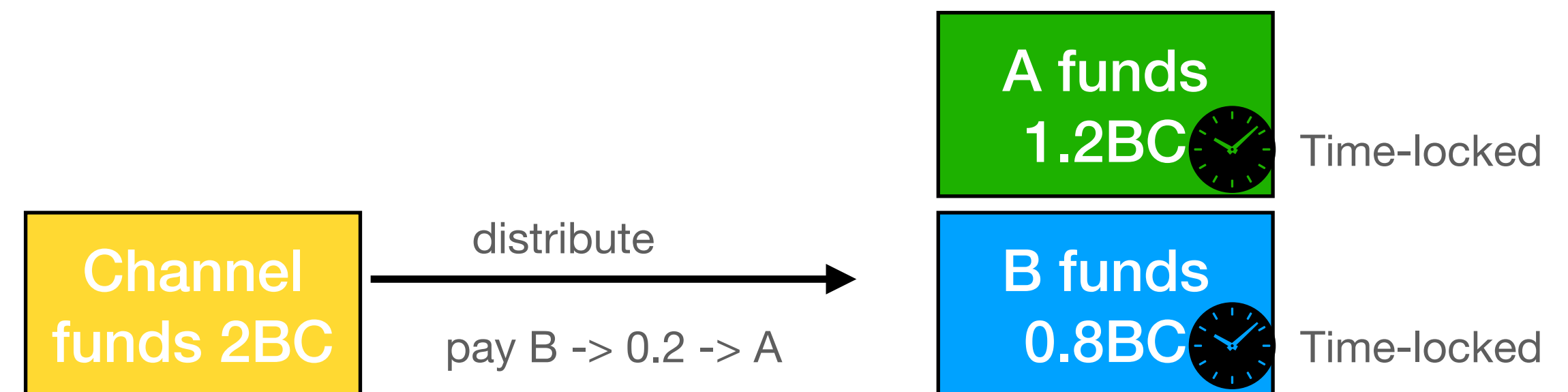
Problem:

- Timeout
- Locked funds

Payment Channels

Example: Lightning channels on UTXO

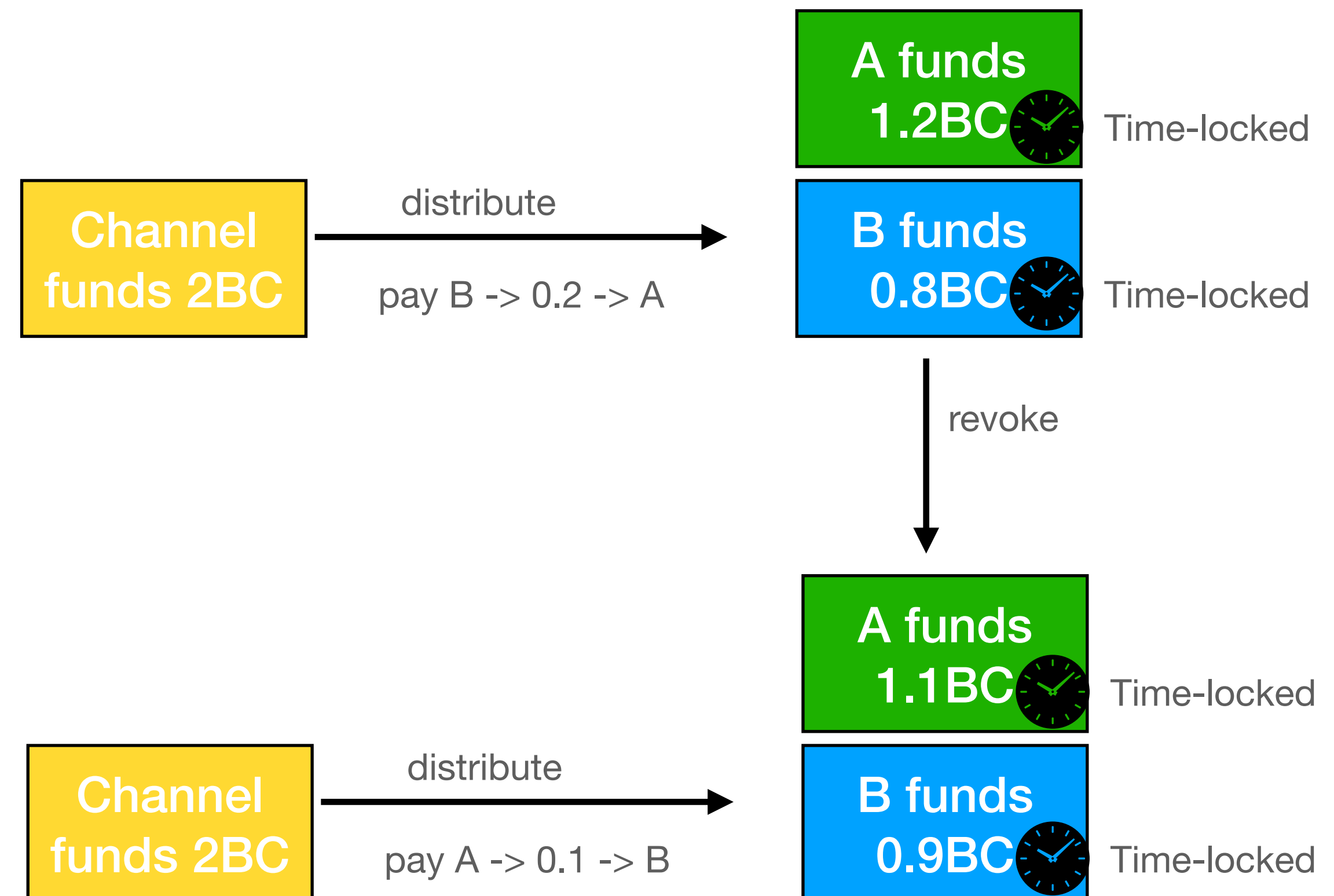
- **Funds are locked in one Output**
- **First channel payment:**
Create valid transaction,
to distribute funds
(not submitted)
- **Second payment**
Create valid transaction to
distribute funds, and
revocation transaction



Payment Channels

Example: Lightning channels on UTXO

- **Funds are locked in one Output**
- **First channel payment:**
Create valid transaction,
to distribute funds
(not submitted)
- **Second payment**
Create valid transaction to
distribute funds, and
revocation transaction

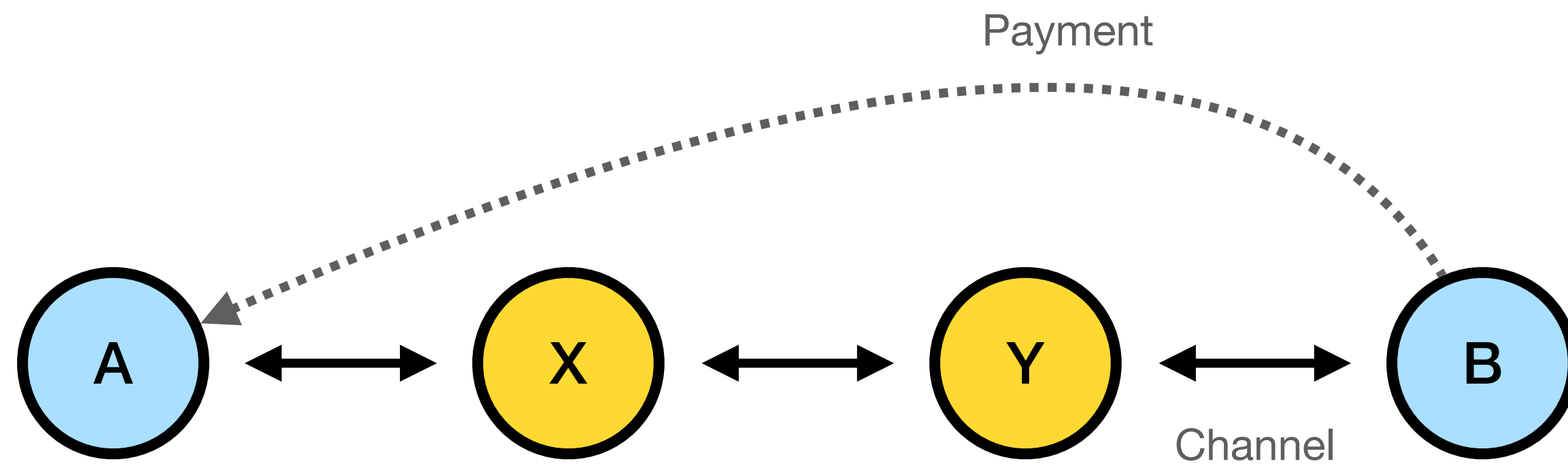


Payment Channels

Example: Multi hop payment

- **Idea:** payment across multiple channels
- Pay fees to intermediates (X and Y)

- **How:** Conditional payments, with secret known to A
- $B \rightarrow Y; Y \rightarrow X; \dots$ s. t. payment is only valid if participants know the secret.
- Friendly settlement: Secret forwarded $A \rightarrow X \rightarrow Y$
- Unfriendly settlement: A publishes secret on chain, X and Y can see secret.



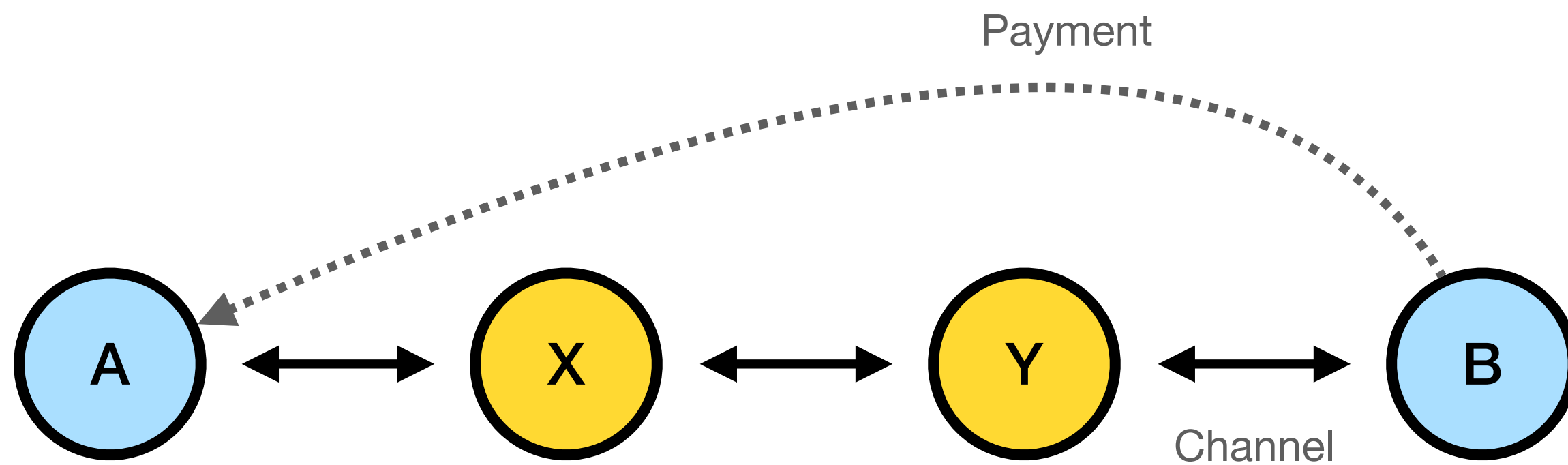
Payment Channels

Example: Payment routing

- Find path from B to A

Problem:

- Limited & changing channel capacity
- Fees play a role
- Privacy of transaction plays a role, *e.g. avoid intermediaries knowing who pays what to whom.*



Payment Channels

Example: Other channels

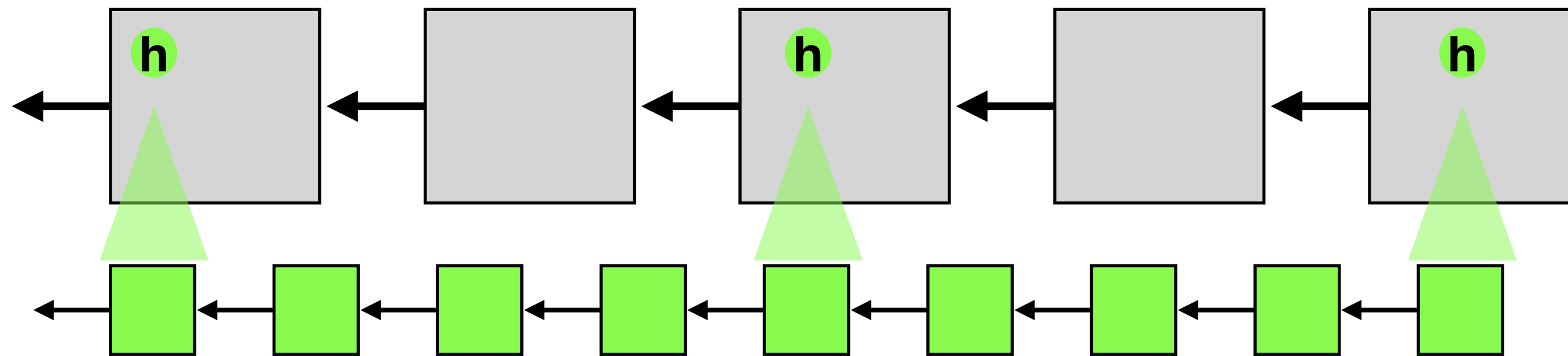
- **Virtual channels:**
 - Given two payment channels $A \leftrightarrow I$ and $I \leftrightarrow B$, create a virtual channel between $A \leftrightarrow B$.
 - Intermediate is only involved in opening and closing the virtual channel.
 - Fewer fees
- **State channels:**
 - A channel where we can create smart contracts.
 - Only channel members can interact with these contracts.

Side Chains

Side Chains

General idea

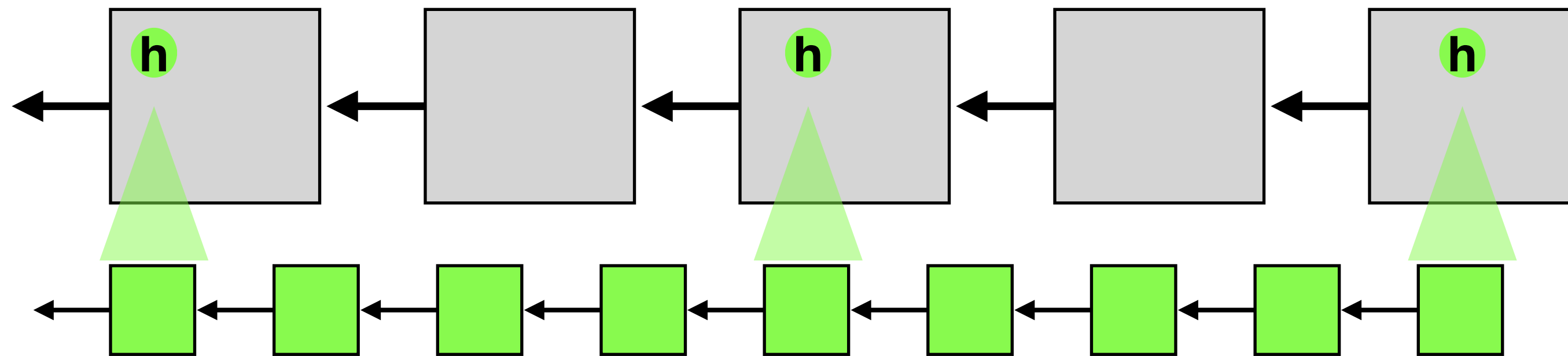
- A separate, smaller blockchain
- State root regularly insterted into main chain



Side Chains

General idea

- A separate, smaller blockchain
- State root regularly insterted into main chain



Benefits:

- State fixed by main chain
- Trusted asset transfer

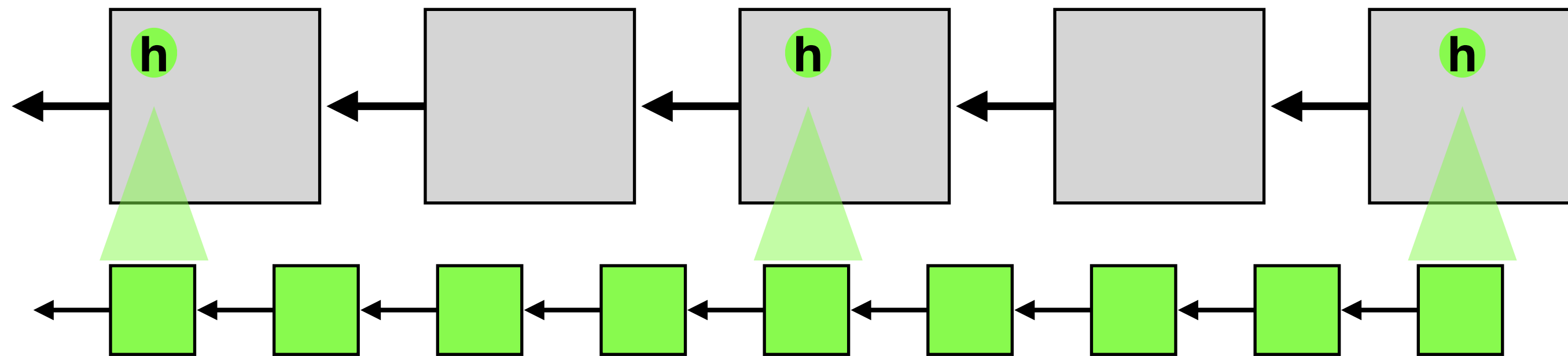
Problem:

- Can a 51% attack on a side chain change state?

Side Chains

General idea

- A separate, smaller blockchain
- State root regularly insterted into main chain



Benefits:

- Side chain may use a different consensus, even single trusted node
- State of sidechain may be private

Problem:

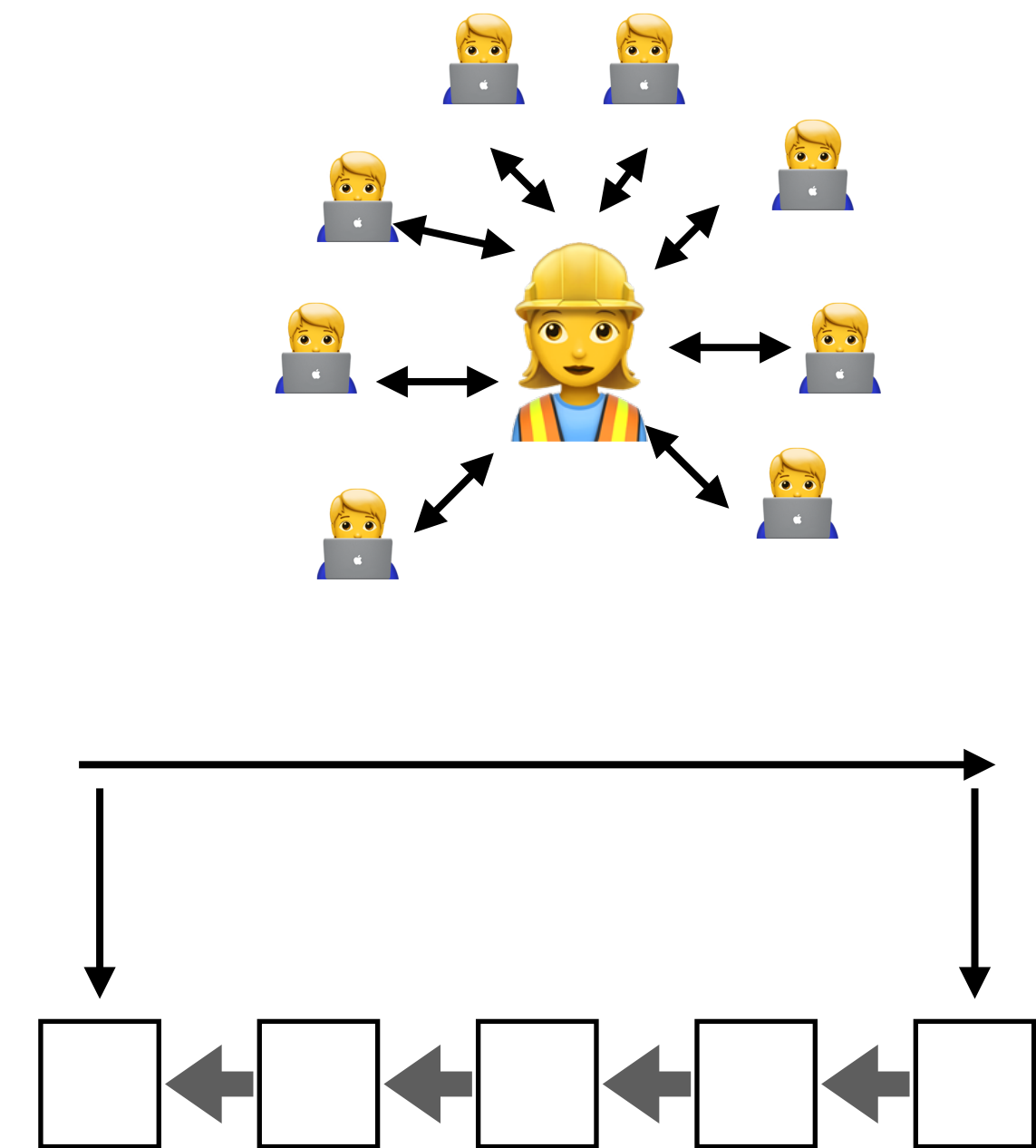
- What can happen?
 - Unavailable?
 - Changed?

Rollups

Rollups

Optimistic rollups

- **A Smart Contract with:**
 - **Balance (all money on the rollup)**
 - **Balances of users (ready to be withdrawn)**
- **Hash of state**
- **Sequencer regularly submits updates to**
 - **Hash of state**
 - **Balances of users**



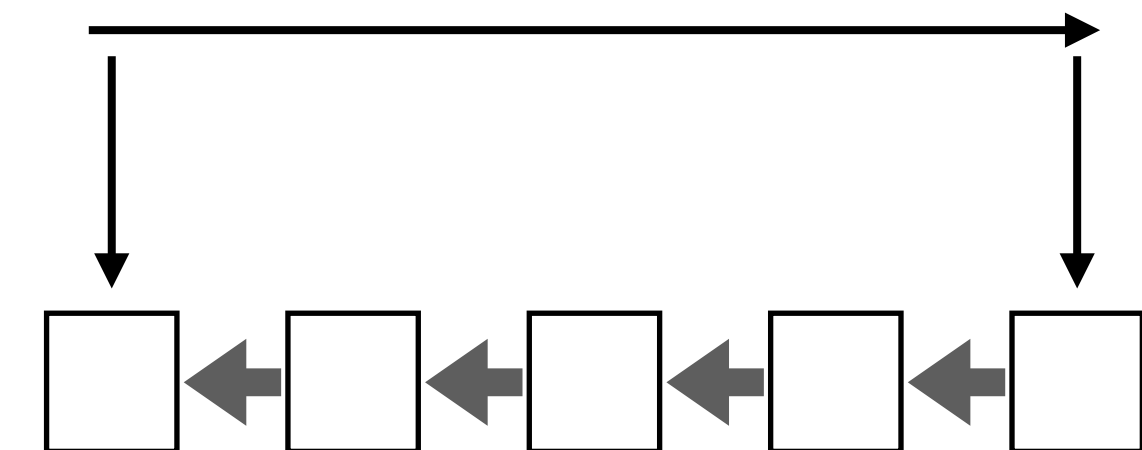
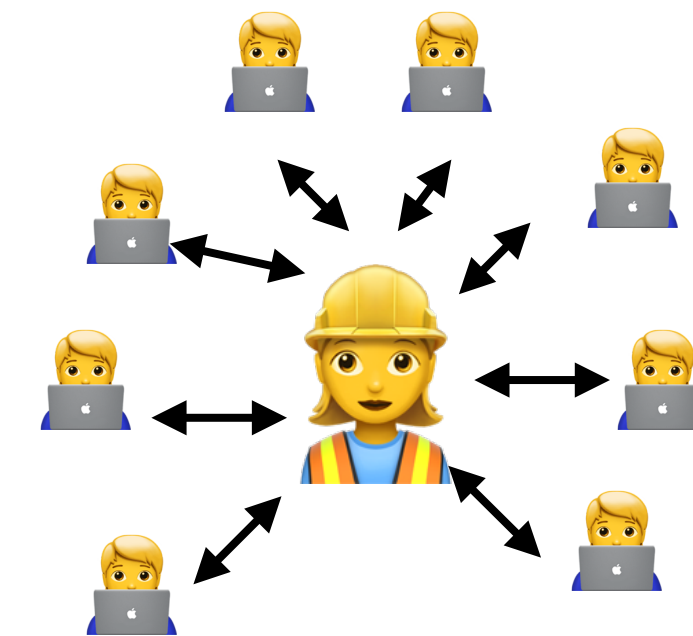
Rollups

Optimistic rollups

```
contract MockRollup {
    bytes32 public stateHash;
    mapping(address => uint256) public balancesToWithdraw;

    // No proof verification: anyone can post an update.
    function update(
        bytes calldata state,
        // transactions: calldata only, not stored
        bytes calldata,
        address[] calldata accounts,
        uint256[] calldata balances
    ) external {
        require(
            accounts.length == balances.length,
            "Length mismatch"
        );

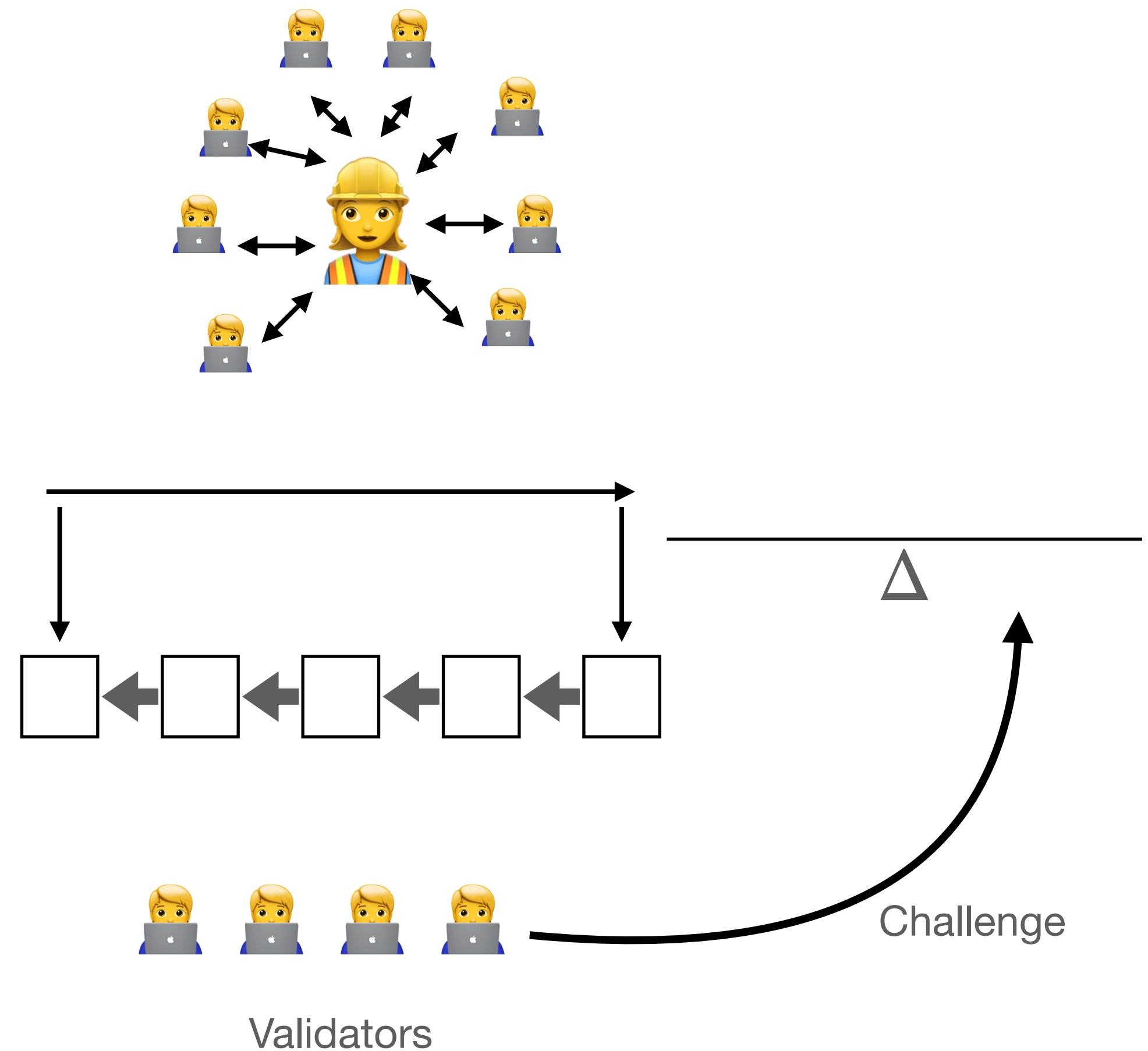
        stateHash = keccak256(state);
        for (uint256 i = 0; i < accounts.length; i++) {
            balancesToWithdraw[accounts[i]] = balances[i];
        }
    }
}
```



Rollups

Optimistic rollups

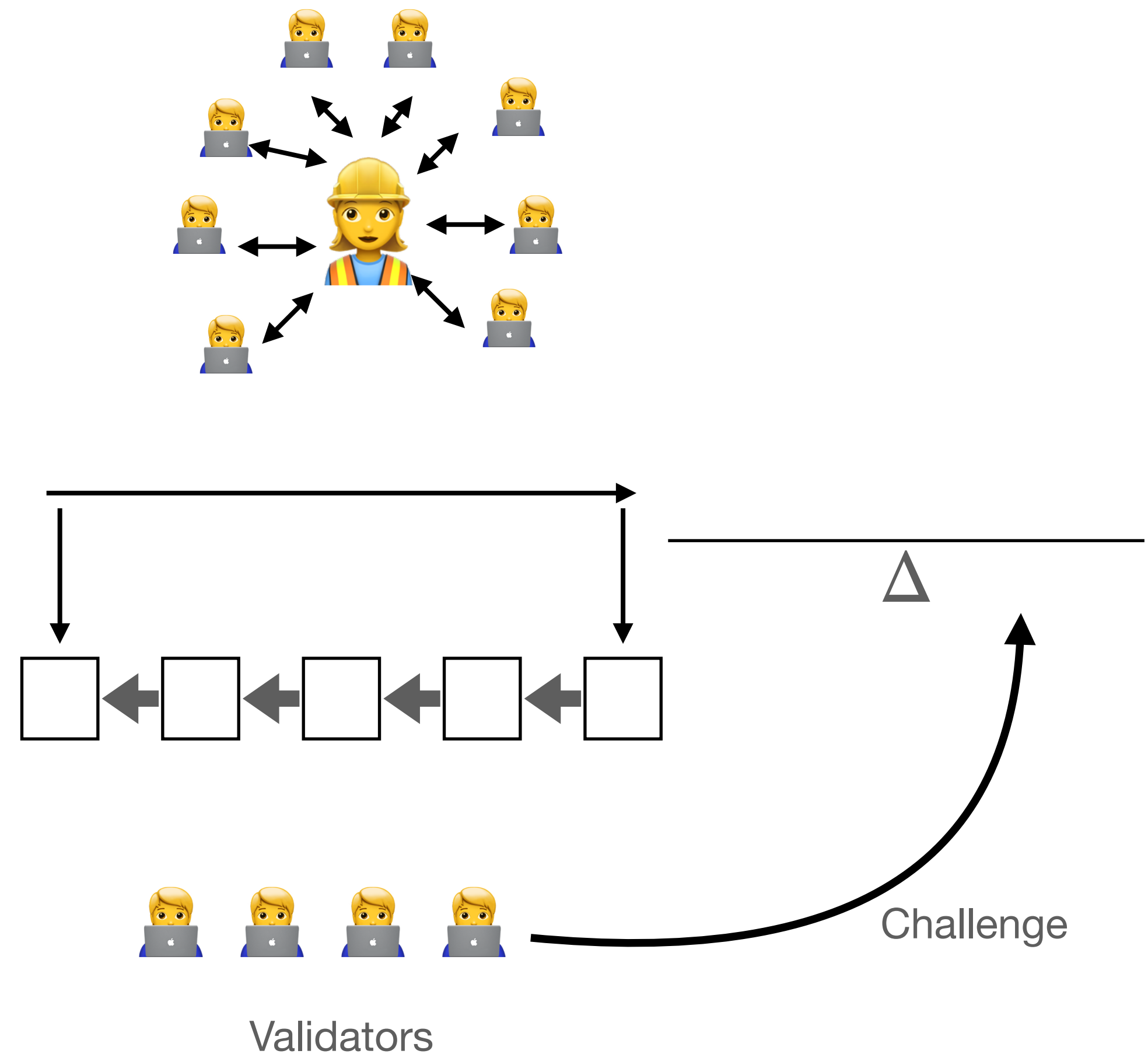
- **Dispute:**
 - **Everyone can recompute and check state**
 - **Validators can submit a challenge**
 - **Update considered stable only if no successful challenge during Δ time**



Rollups

Optimistic rollups

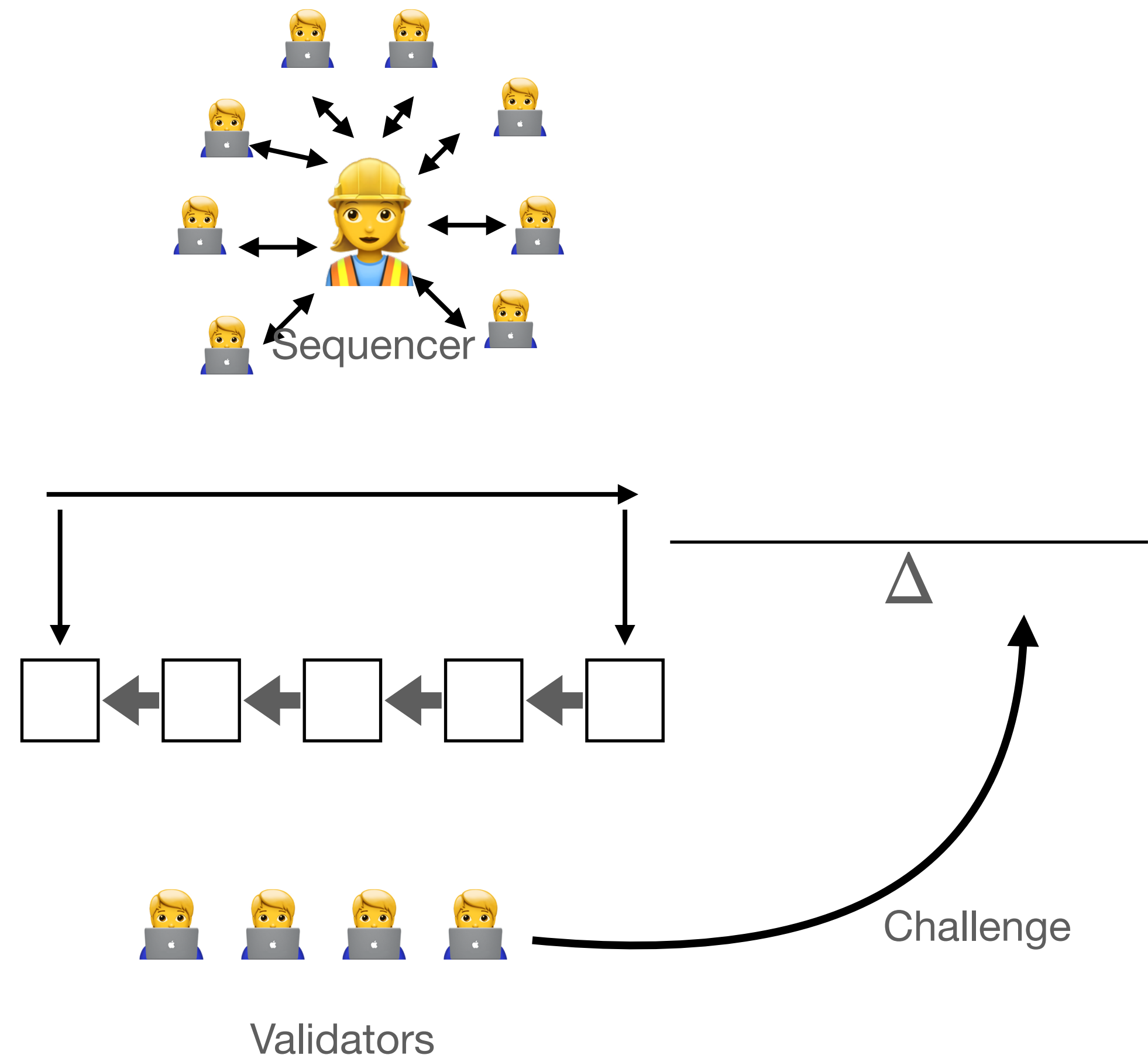
- **Challenge:**
 - **Everyone can recompute and check state**
 - **Validators can submit a challenge**
 - **Update considered stable only if no successful challenge during Δ time**



Rollups

Optimistic rollups

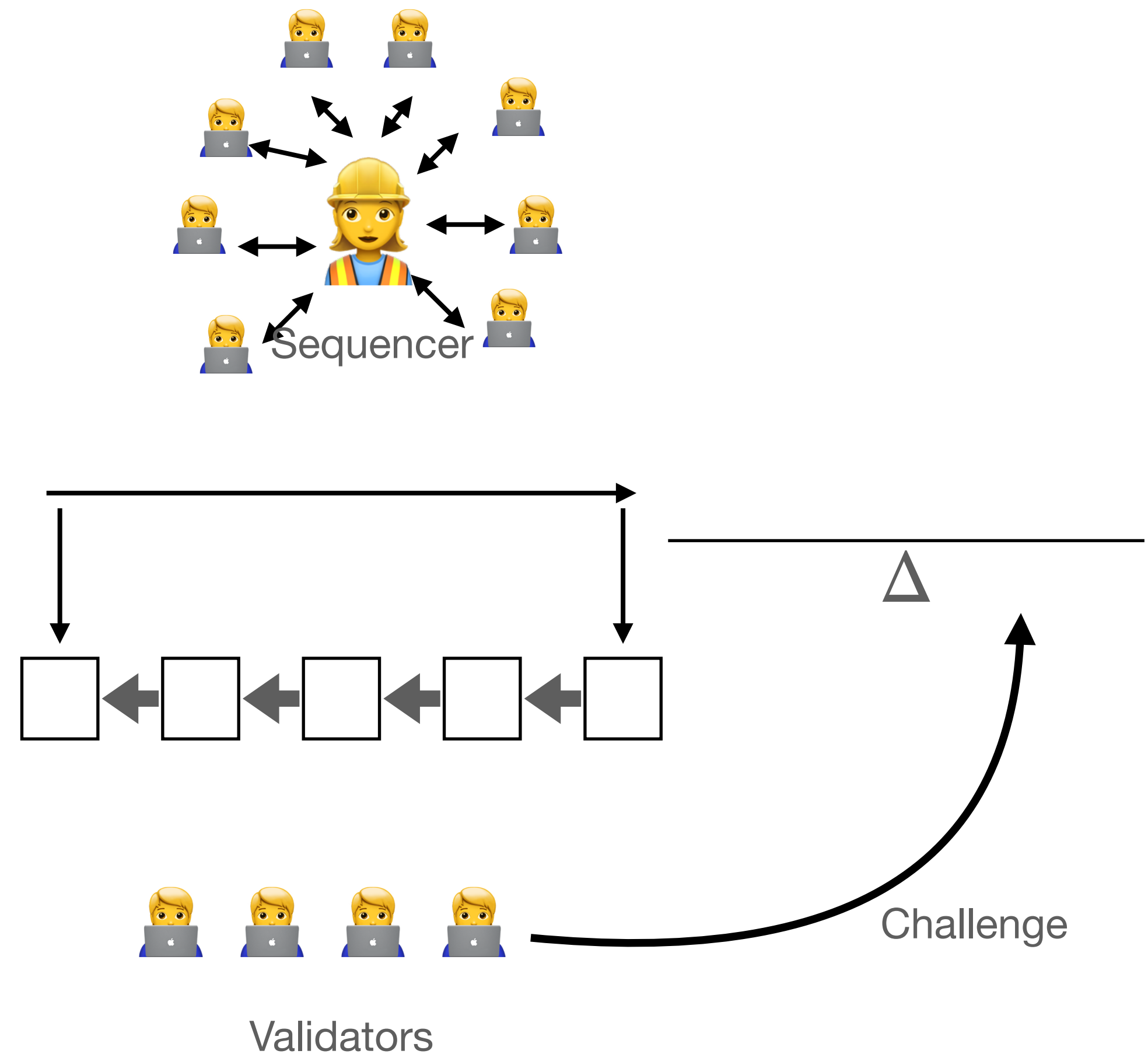
- **Challenge example**
 - **Validator points to faulty executed transaction**
 - **Sequencer provides pre and post state of transaction**
 - **Transaction re-executed in smart contract, to decide on challenge**



Rollups

Optimistic rollups

- Who can be sequencer?
 - Incentivised?
- Who can be validator?
 - Incentivised?
- How long is the challenge period?

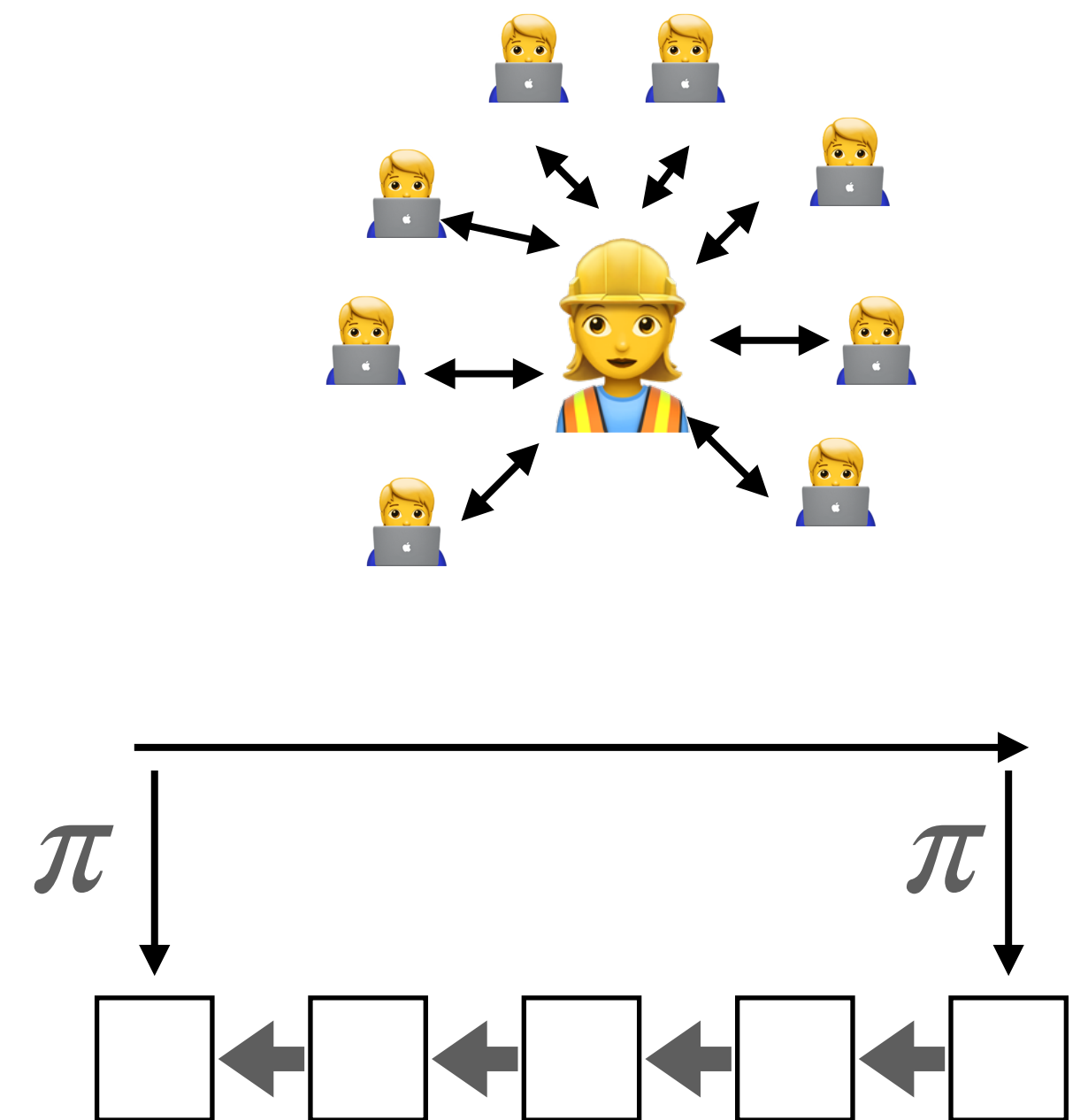


Commit chains

zkRollups

What is submitted to the blockchain?

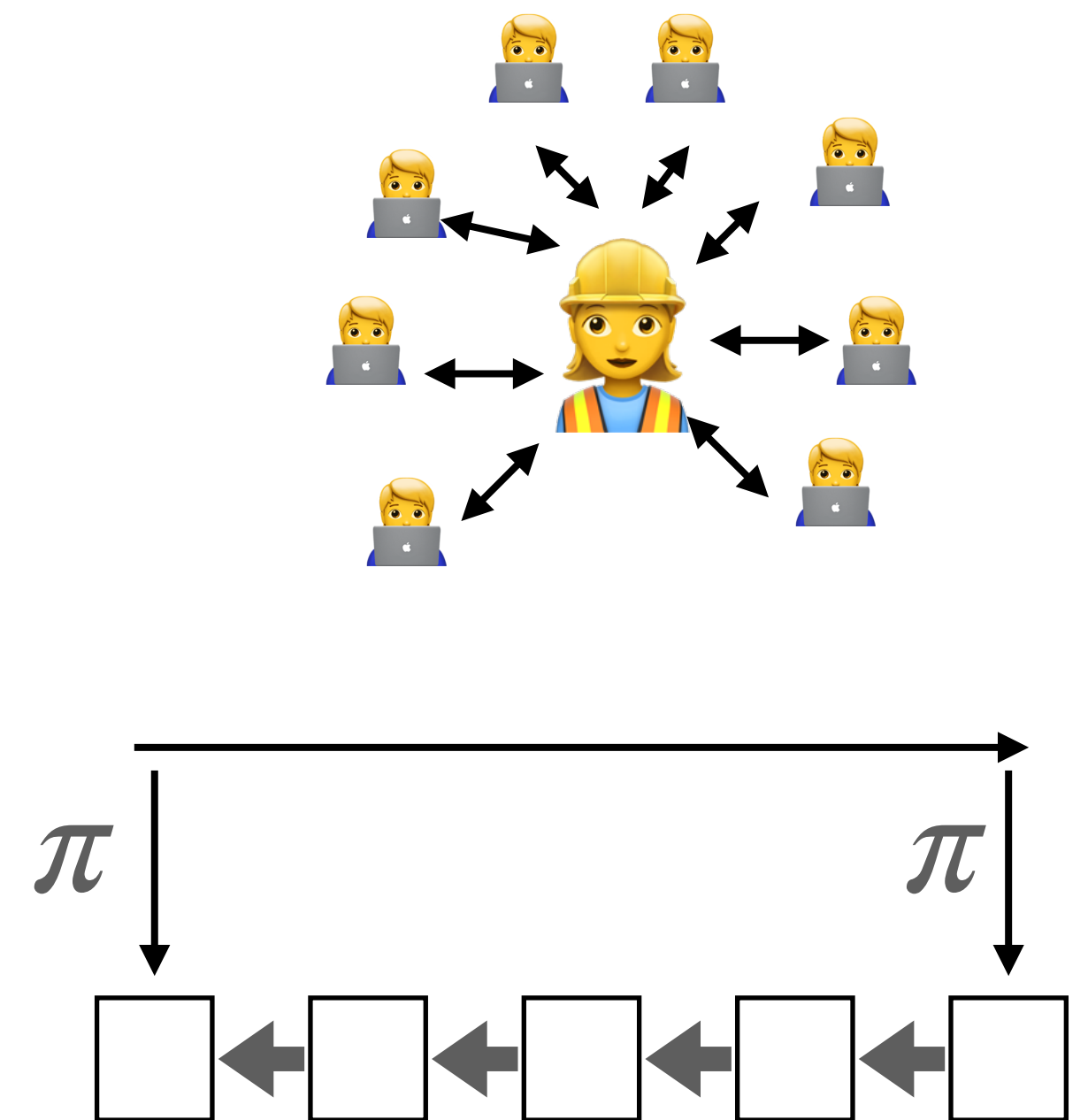
- Sequencer submits zero-knowledge proof:
 - “I know tx , s_1 , s_2 such that:
 - $H(s_1)$ = old state hash
 - $H(s_2)$ = new state hash
 - $s_1 \cdot apply(tx) = s_2$ ”
- Proof verified in Smart Contract



Commit chains

zkRollups

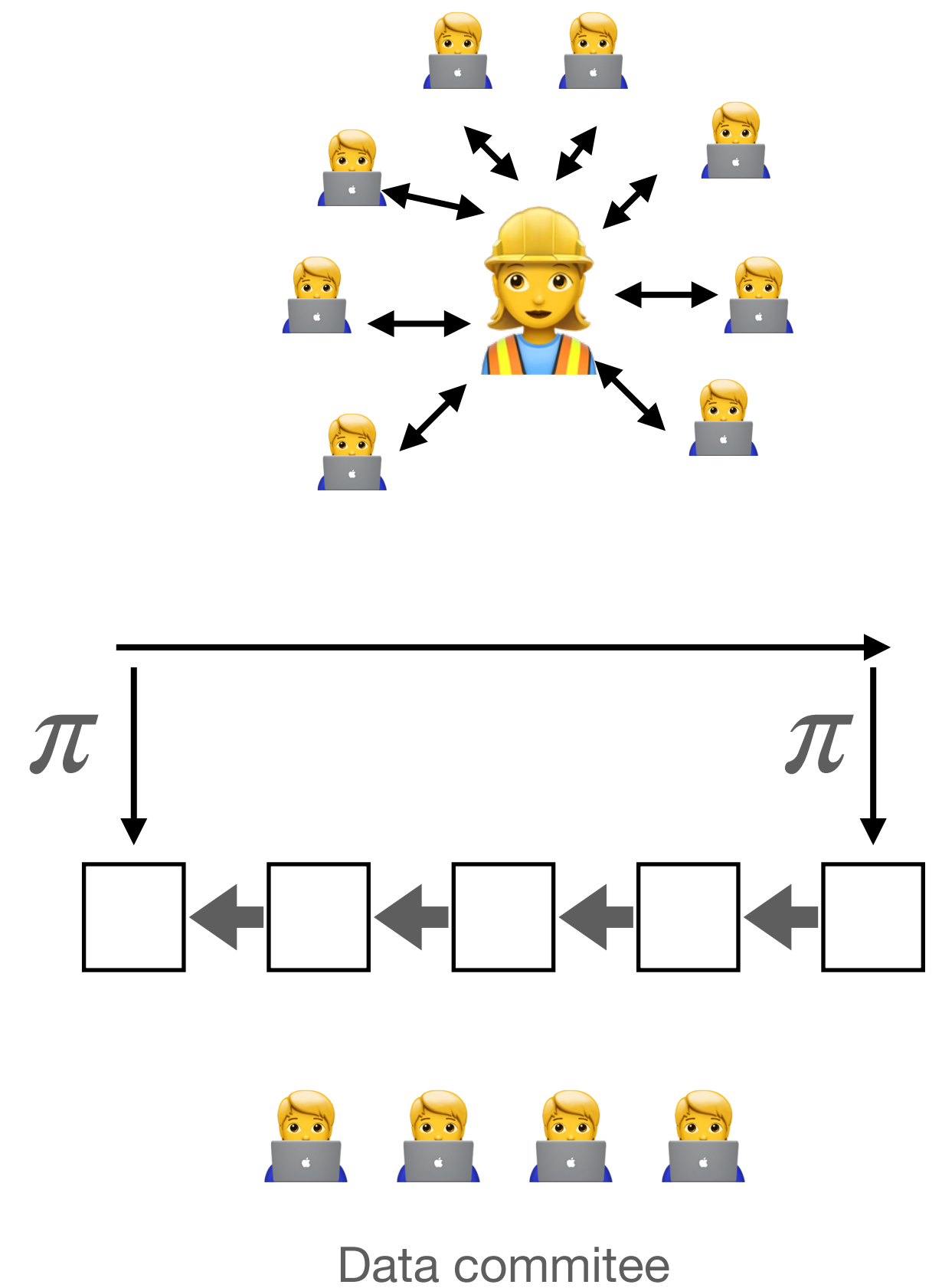
- zkProof allows:
 - no Dispute
 - no waiting for dispute before withdraw
 - need to compute proof
 - need to verify in smart contract



Commit chains

Validium

- Rollup without putting data on chain
 - Less data sent and stored



Payment channels

Payment channels

General idea

- **Idea:** If two parties agree, they can do a transaction outside of the chain without paying fees.
 - Once they disagree, they can use the chain to settle the dispute.
 - Can increase transaction throughput
 - Can give low fees

Payment Channels

Example: Uni-directional payment channel

- **Idea:** Allow any number of payments from A to B within given limit
- A creates contract with balance.
- A can send signed statements of B's balance to B
- B can cash in his balance with the contract
- If B does not cash in, A can terminate the contract and get back the balance, after expiration date.

Check example

<https://solidity-by-example.org/app/uni-directional-payment-channel/>

Payment Channels

Example: Bi-directional payment channel

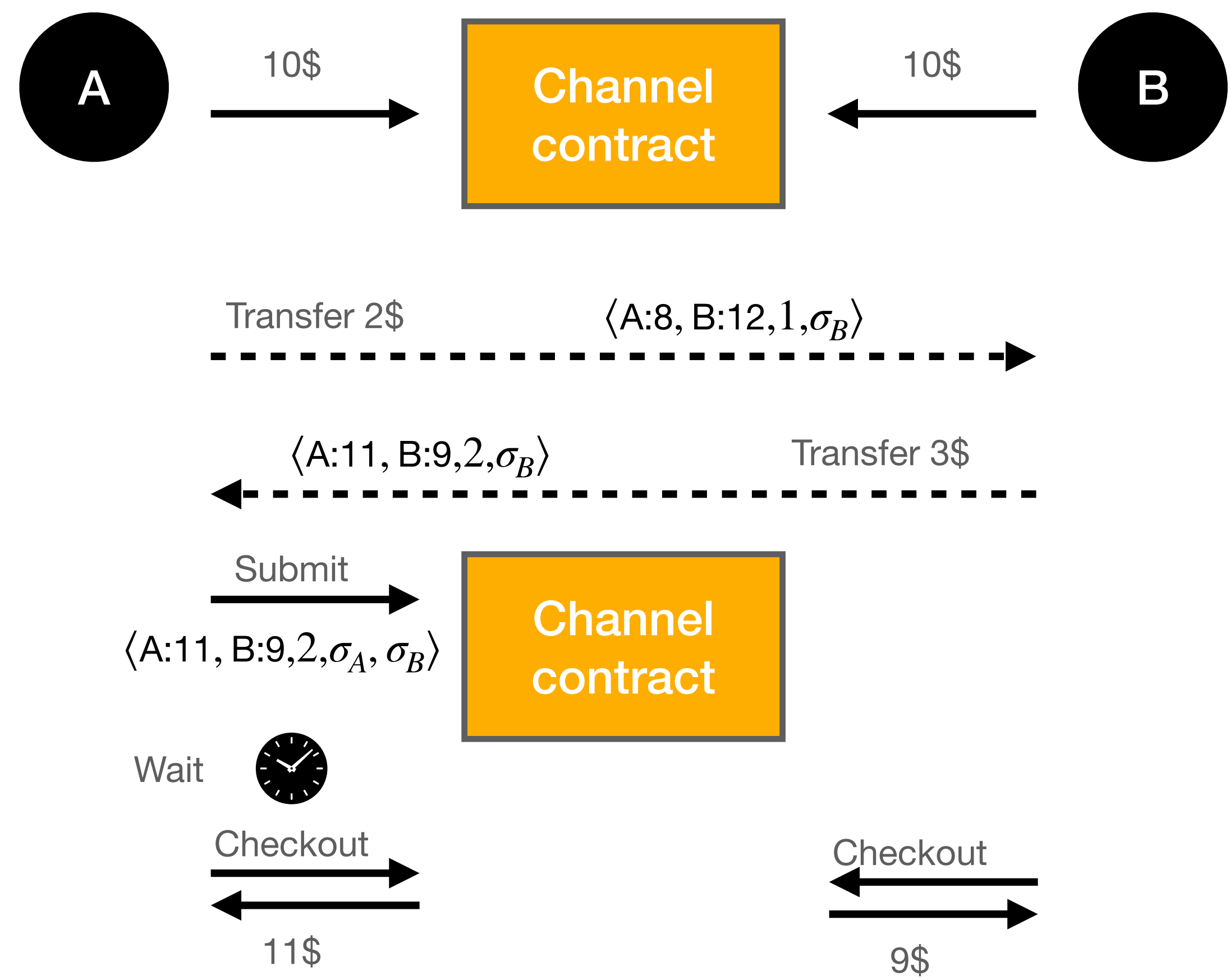
- **Idea:** Allow any number of payments between A and B within given limit,
- A and B both pay a balance to contract
- A and B can send signed statements of their balances to each other, with increasing nonces
- A or B can submit balance, signed by both to contract. This triggers countdown
- If other party does not submit a balance with larger nonce, balances are paid out.

Check example

<https://solidity-by-example.org/app/bi-directional-payment-channel/>

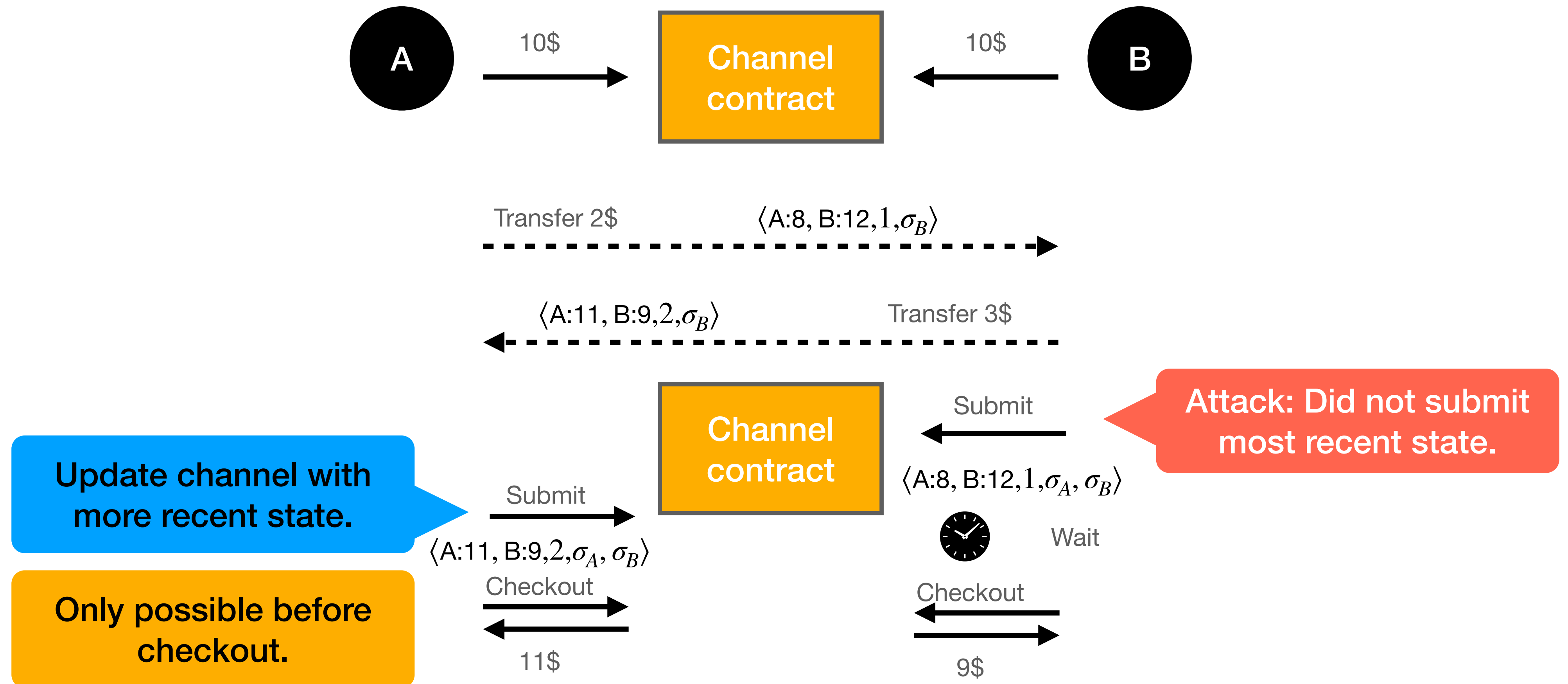
Payment Channels

Example: Bi-directional payment channel



Payment Channels

Example: Bi-directional payment channel



Payment Channels

Example: Bi-directional payment channel

- **Idea:** Allow any number of payments between A and B within given limit,

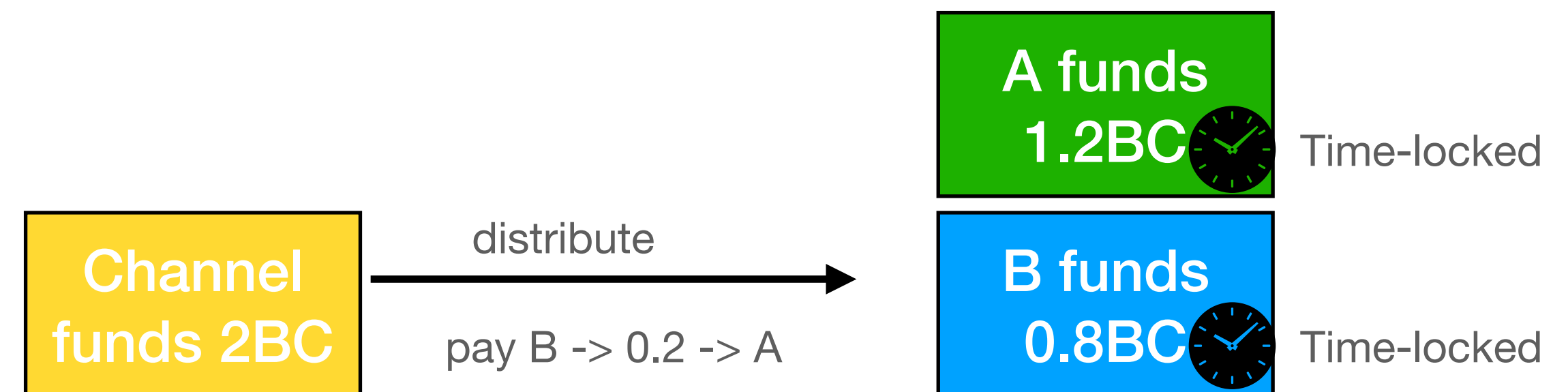
Problem:

- Timeout
- Locked funds

Payment Channels

Example: Lightning channels on UTXO

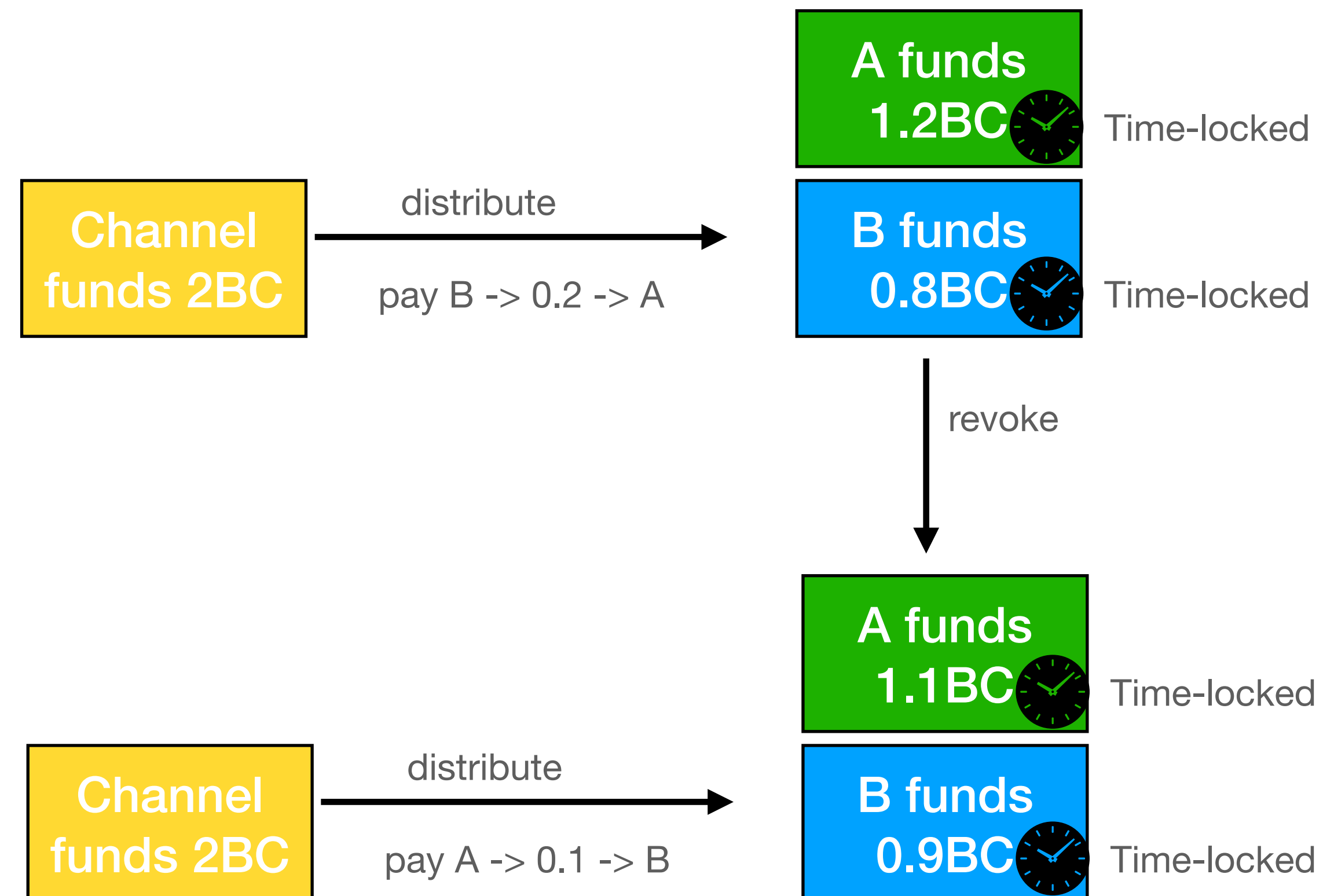
- **Funds are locked in one Output**
- **First channel payment:**
Create valid transaction,
to distribute funds
(not submitted)
- **Second payment**
Create valid transaction to
distribute funds, and
revocation transaction



Payment Channels

Example: Lightning channels on UTXO

- **Funds are locked in one Output**
- **First channel payment:**
Create valid transaction,
to distribute funds
(not submitted)
- **Second payment**
Create valid transaction to
distribute funds, and
revocation transaction

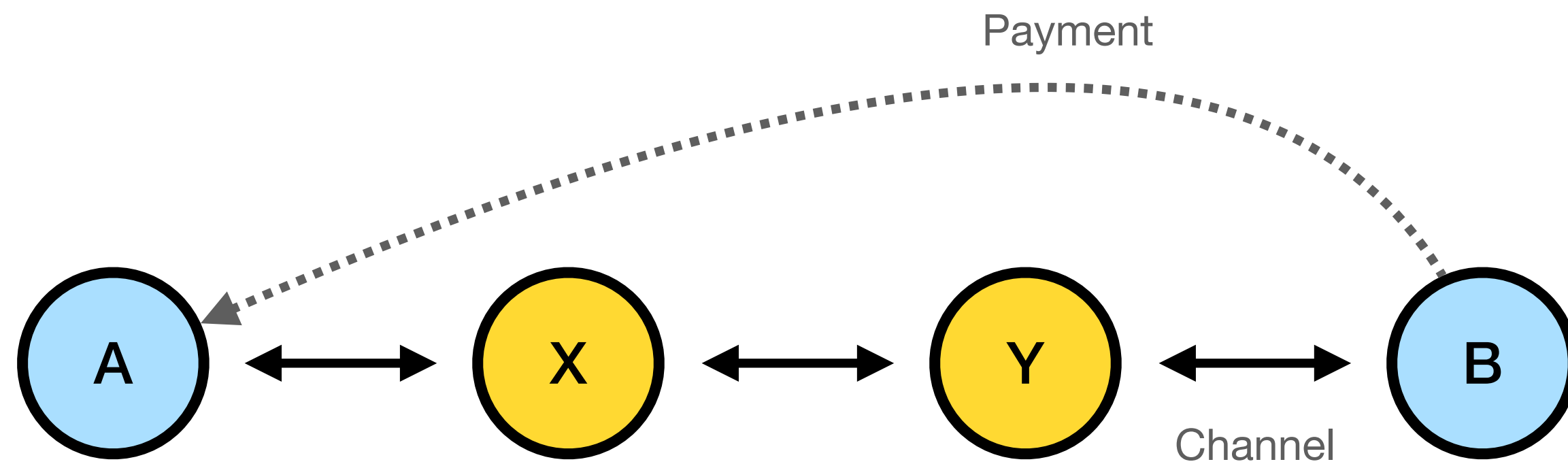


Payment Channels

Example: Multi hop payment

- **Idea:** payment across multiple channels
- Pay fees to intermediates (X and Y)

- **How:** Conditional payments, with secret known to A
- $B \rightarrow Y$; $Y \rightarrow X$; ... s. t. payment is only valid if participants know the secret.
- Friendly settlement: Secret forwarded $A \rightarrow X \rightarrow Y$
- Unfriendly settlement: A publishes secret on chain, X and Y can see secret.



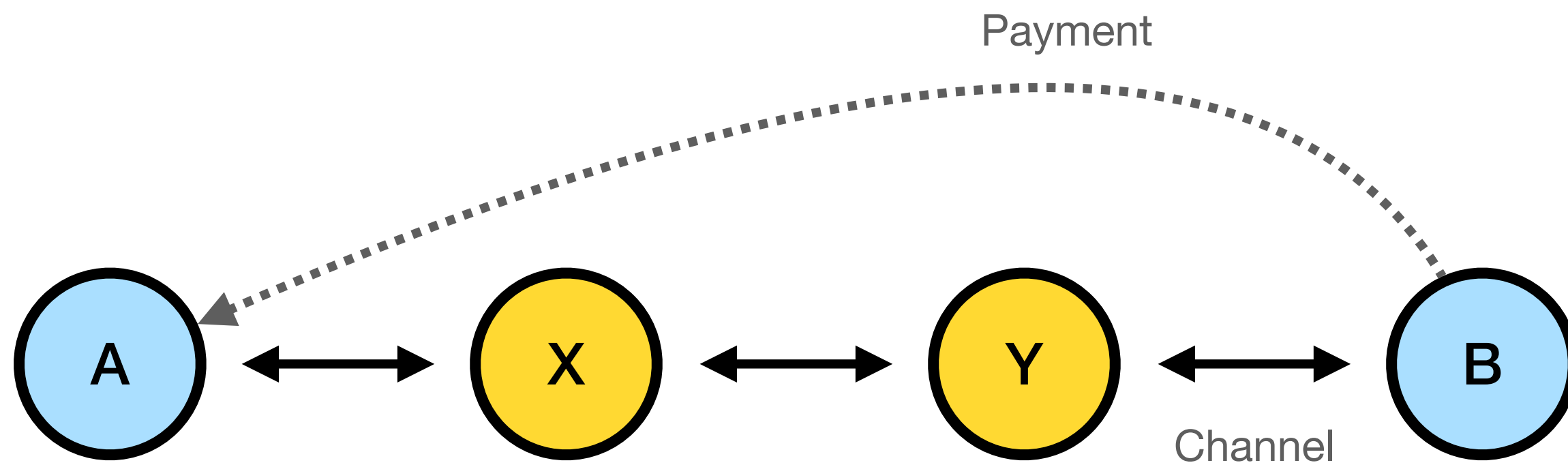
Payment Channels

Example: Payment routing

- Find path from B to A

Problem:

- Limited & changing channel capacity
- Fees play a role
- Privacy of transaction plays a role, *e.g. avoid intermediaries knowing who pays what to whom.*



Payment Channels

Example: Other channels

- **Virtual channels:**
 - Given two payment channels $A \leftrightarrow I$ and $I \leftrightarrow B$, create a virtual channel between $A \leftrightarrow B$.
 - Intermediate is only involved in opening and closing the virtual channel.
 - Fewer fees
- **State channels:**
 - A channel where we can create smart contracts.
 - Only channel members can interact with these contracts.

Off Chain comparison

	On chain transaction	Channel	Commit chain
Cheep fees			
Fast confirmation			
Can go offline			
Unlimited capacity			
Joining	Not necessary	Setup cost	No cost